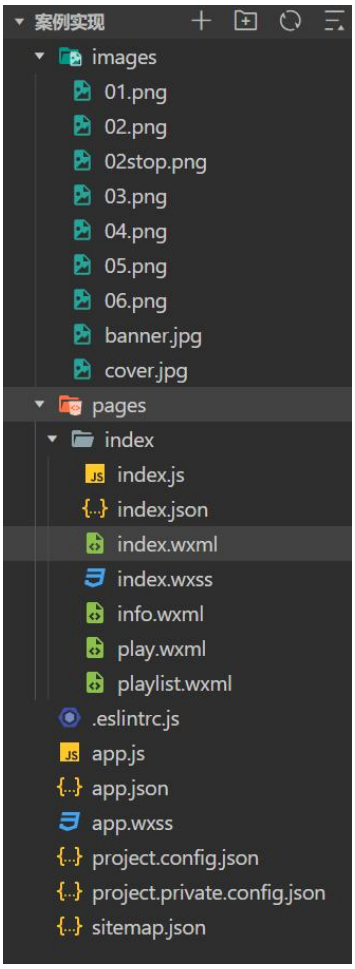
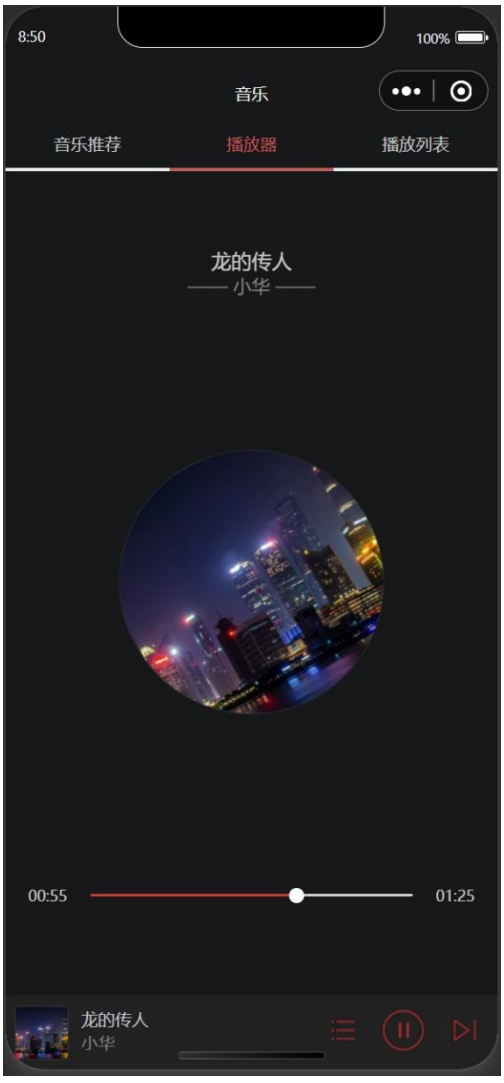
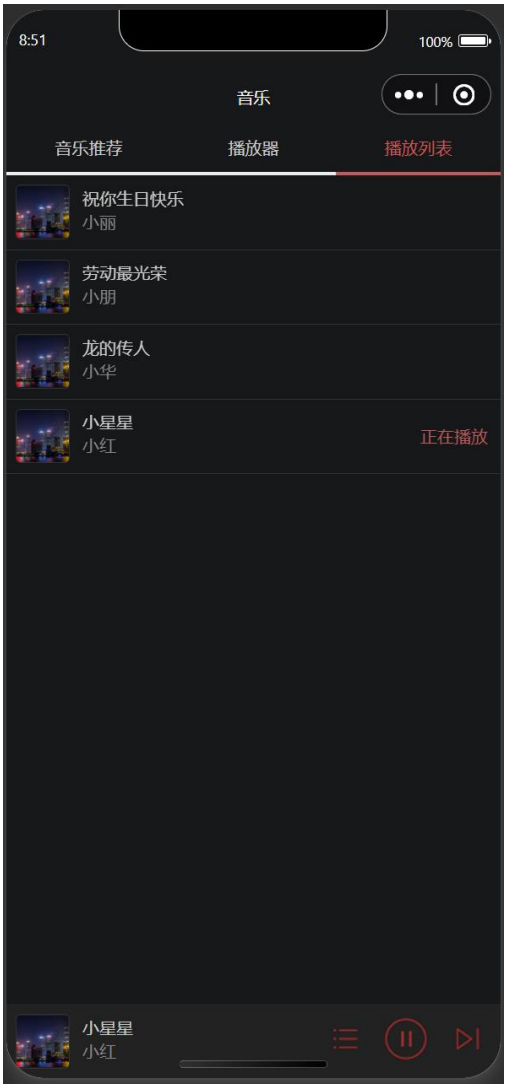
案例 1：音乐播放器


案例实现
images
01.png
02.png
02stop.png
03.png
04.png
05.png
06.png
banner.jpg
cover.jpg
pages
index
index.js
index.json
index.wxml
index.wxss
info.wxml
play.wxml
playlist.wxml
.eslintrc.js
app.js
app.json
app.wxss
project.config.json
project.private.config.json
sitemap.json


8:49
100%
音乐
音乐推荐
播放器
播放列表
私人FM
26
每日歌曲推荐
云音乐新歌榜
推荐歌曲
山水之间
惊鸿一面
稻香
如果当时
清明雨上
有何不可
劳动最光荣
小朋


8:50
100%
音乐
音乐推荐
播放器
播放列表
龙的传人
小华
00:55
01:25
龙的传人
小华


8:51
100%
音乐
音乐推荐
播放器
播放列表
祝你生日快乐
小丽
劳动最光荣
小朋
龙的传人
小华
小星星
小红
正在播放
小星星
小红

案例实现

1. 准备工作

在开发本案例前，需要先完成一些准备工作，主要包括创建项目、配置页面、配置导航栏、复制素材和启动服务器，具体步骤如下。

① 创建项目。在微信开发者工具中创建一个新的微信小程序项目，项目名称为“音乐播放器”，模板选择“不使用模板”。

② 配置页面。本项目中的页面文件如表 4-4 所示。

表 4-4　本项目中的页面文件

文件路径	说明
pages/index/index.js	index 页面的逻辑文件
pages/index/index.json	index 页面的配置文件
pages/index/index.wxss	index 页面的样式文件
pages/index/index.wxml	index 页面的结构文件
pages/index/info.wxml	“音乐推荐”标签页的结构文件
pages/index/play.wxml	“播放器”标签页的结构文件
pages/index/playlist.wxml	“播放列表”标签页的结构文件

在表 4-4 中，实际只有一个页面，即 pages/index/index 页面，整体的页面结构在 index.wxml 文件中编写。由于内容区域中 3 个标签页的内容较多，为了避免页面嵌套层级过多，所以拆分到 info.wxml、play.wxml、playlist.wxml 文件中，使代码容易阅读、便于维护。

③ 配置导航栏。在 pages/index/index.json 文件中配置页面导航栏，具体代码如下。

```
{
  "navigationBarTitleText": "音乐",
  "navigationBarBackgroundColor": "#17181a",
  "navigationBarTextStyle": "white"
}
```

上述代码将导航栏标题设置为“音乐”，导航栏背景颜色为#17181a，文字颜色为 white。“音乐”导航栏的效果如图 4-7 所示。

④ 复制素材。从本书配套资源中找到本案例，复制以下素材到本项目中。

- pages/index/index.wxss 文件，该文件中保存了本项目的页面样式素材。
- images 文件夹，该文件夹保存了本项目所用的图片素材。

上述步骤操作完成后，“音乐播放器”微信小程序的目录结构如图 4-8 所示。

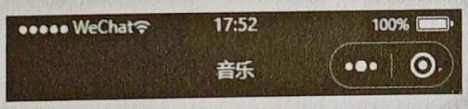


图4-7　“音乐”导航栏的效果

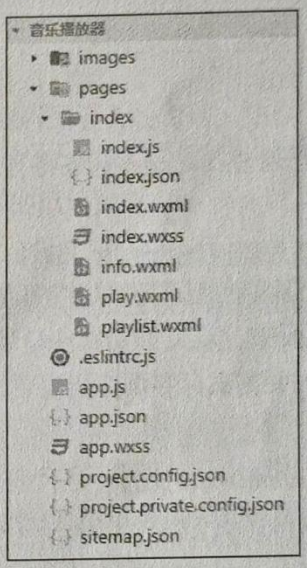


图4-8　“音乐播放器”微信小程序的目录结构

⑤ 启动服务器。从本书配套资源中找到本案例的源代码，进入“服务器端”文件夹。该文件夹下的内容为 Node.js 本地 HTTP 服务器程序。打开命令提示符，切换工作目录到当前目录，然后在命令提示符中执行如下命令，启动服务器。

```
node index.js
```

至此，准备工作已经全部完成。

2. 实现“音乐播放器”微信小程序的页面结构

在 pages/index/index.wxml 文件中编写“音乐播放器”微信小程序的页面结构，具体步骤如下。

① 编写标签栏区域的页面结构，具体代码如下。

```
1 <view class="tab">
2   <view class="tab-item">音乐推荐</view>
3   <view class="tab-item">播放器</view>
4   <view class="tab-item">播放列表</view>
5 </view>
```

在上述代码中，第 2～4 行代码定义了 3 个 view 组件，分别用于实现“音乐推荐”标签按钮、“播放器”

标签按钮和“播放列表”标签按钮。

② 使用swiper组件实现标签页切换。虽然通过配置tabBar配置项的属性也可以实现标签页切换，但考虑到swiper组件的切换效果更流畅、用户体验更好，因此本项目使用swiper组件实现标签页切换效果。接下来编写内容区域的页面结构，具体代码如下。

```
1 <view class="content">
2   <swiper>
3     <swiper-item>
4       <!-- 音乐推荐 -->
5       <include src="info.wxml" />
6     </swiper-item>
7     <swiper-item>
8       <!-- 播放器 -->
9       <include src="play.wxml" />
10    </swiper-item>
11    <swiper-item>
12      <!-- 播放列表 -->
13      <include src="playlist.wxml" />
14    </swiper-item>
15  </swiper>
16 </view>
```

在上述代码中，第2～15行代码定义了swiper组件，用于实现标签页的切换。其中，第5行、第9行和第13行代码通过<include>标签将音乐推荐、播放器和播放列表这3个标签页对应的WXML文件引入当前文件中。

③ 编写播放器区域的结构，具体代码如下。

```
<view class="player"></view>
```

在上述代码中，定义了1个view组件，用于展示播放器区域。

④ 为了方便测试标签页切换效果，在每个标签页文件中编写一些简单的代码。

pages/index/info.wxml文件中的代码如下。

```
1 <view style="background: #ccc; height: 100%; color: #000;">
2   info
3 </view>
```

pages/index/play.wxml文件中的代码如下。

```
1 <view style="background: #ccc; height: 100%; color: #000;">
2   play
3 </view>
```

pages/index/playlist.wxml文件中的代码如下。

```
1 <view style="background: #ccc; height: 100%; color: #000;">
2   playlist
3 </view>
```

完成上述代码后，“音乐播放器”微信小程序的页面结构效果如图4-9所示。

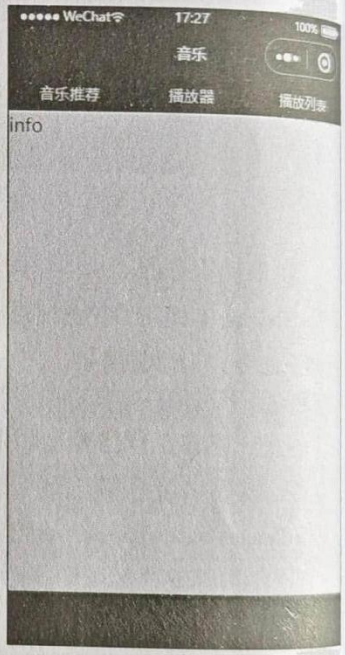


图4-9 “音乐播放器”微信小程序的页面结构效果

3. 实现标签页切换

在“音乐播放器”微信小程序中，可以通过点击标签栏区域的标签按钮切换到对应的标签页，具体实现步骤如下。

① 在pages/index/index.wxml文件中修改标签栏区域，具体代码如下。

```
1 <view class="tab-item" bindtap="changeItem" data-item="0">音乐推荐</view>
```

```
<view class="tab-item" bindtap="changeItem" data-item="1">播放器</view>
<view class="tab-item" bindtap="changeItem" data-item="2">播放列表</view>
```

在上述代码中，给每个 view 组件绑定 tap 事件，事件处理函数为 changeItem()；设置了 data-item 自定义属性，表示按下标签栏区域中标签按钮时的值分别为 0、1、2，这 3 个数字是 swiper 组件中对应的 swiper-item 组件的索引。

② 修改内容区域，具体代码如下。

```
<swiper current="{{ item }}"></swiper>
```

在上述代码中，给 swiper 组件添加了 current 属性，属性值为 item，表示当前显示索引为 item 的 swiper-item 组件。

③ 在 pages/index/index.js 文件中编写页面中所需的数据和 changeItem()事件处理函数，具体代码如下。

```
Page({
  data: {
    item: 0
  },
  changeItem: function (e) {
    this.setData({
      item: e.target.dataset.item
    })
  }
})
```

在上述代码中，第 3 行代码定义了属性 item，属性值为 0，表示默认显示 swiper 组件中的第 1 个 swiper-item 组件；第 5～9 行代码定义了 changeItem()事件处理函数，将页面中 item 的值设置为 data-item 自定义属性的值。

上述代码完成后，即可实现点击标签按钮切换到对应的标签页的功能。

④ 在切换标签页后，还需要改变当前标签页对应的标签栏中标签按钮的样式，将当前标签页对应的标签按钮设为 active 样式。为此，需要修改 pages/index/index.wxml 文件中的内容区域，通过给 swiper 组件绑定 change 事件感知到标签页的变化，具体代码如下。

```
<swiper current="{{ item }}" bindchange="changeTab">
```

在上述代码中，给 swiper 组件绑定的事件处理函数为 changeTab()，该函数会在 swiper 组件的 current 属性发生变化即标签页切换时调用。

⑤ 在 pages/index/index.js 文件的 data 数据中添加 tab 属性，并通过 changeTab()事件处理函数设置 tab 属性的值，具体代码如下。

```
data: {
  item: 0,
  tab: 0
},
changeTab: function (e) {
  this.setData({
    tab: e.detail.current
  })
}
```

在上述代码中，tab 属性的初始值为 0，表示默认情况下应为第 1 个标签按钮添加 active 样式。第 5～9 行代码为 changeTab()事件处理函数，其中，第 6～8 行代码用于将 tab 属性的值更改为当前切换的标签页的索引。

⑥ 修改 pages/index/index.wxml 文件中的标签栏区域，具体代码如下。

```
<view class="tab-item {{ tab == 0 ? 'active' : '' }}" bindtap="changeItem" data-item="0">音乐推荐</view>
<view class="tab-item {{ tab == 1 ? 'active' : '' }}" bindtap="changeItem" data-item="1">播放器</view>
```

```
3 <view class="tab-item {{ tab == 2 ? 'active' : '' }}" bindtap="changeItem" data-item
"2">播放列表</view>
```

在上述代码中，通过判断 tab 属性的值，实现为当前标签按钮添加 active 样式。

完成上述代码后，标签页的切换功能已经实现。

4. 实现“音乐推荐”标签页

“音乐推荐”是内容区域中的第 1 个标签页，对应 swiper-item 组件的索引为 0，该页面由 3 个部分组成，分别是轮播图、功能按钮、推荐歌曲。由于内容区域的可视高度是有限的，而“音乐推荐”标签页的实际页面内容可能会超出内容区域的可视高度，因此需要将“音乐推荐”标签页放入可滚动的容器中，具体实现步骤如下。

① 在 pages/index/info.wxml 文件中删除原有代码，然后创建一个滚动区域，具体代码如下。

```
1 <scroll-view scroll-y class="content-info">
2   <view style="height: 1000px;background: #fff;"></view>
3   <view style="background: #ccc;color: red;">到达底部</view>
4 </scroll-view>
```

在上述代码中，第 1～4 行代码定义了 scroll-view 组件，实现当实际内容高度超过内容区域的显示范围后，可以上下滚动的效果。其中，第 2 行代码定义了 1 个 view 组件，高度为 1000px，用于将内容区域的页面撑起来，从而使滚动条出现；第 3 行代码用于提示当前是否已经滚动到底部。

完成上述代码，运行程序，将滚动区域滚动到底部，页面效果如图 4-10 所示。

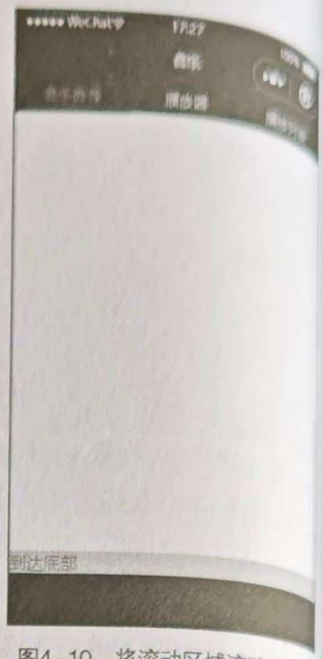


图4-10 将滚动区域滚动到底部

从图 4-10 可以看出，当前滚动区域已经可以滚动，并且当滚动到达底部时，底部“到达底部”内容可以正确显示出来。

② “音乐推荐“标签页的顶部区域为轮播图，切换效果通过 swiper 组件实现。在 pages/index/info.wxml 文件中编写轮播图的页面结构，首先需删除原有代码，然后再进行编写，具体代码如下。

```
1 <swiper class="content-info-slide" indicator-color="rgba(255,255,255,.5)" indicator-
active-color="#fff" indicator-dots circular autoplay>
2   <swiper-item>
3     <image src="/images/banner.jpg" />
4   </swiper-item>
5   <swiper-item>
6     <image src="/images/banner.jpg" />
7   </swiper-item>
8   <swiper-item>
9     <image src="/images/banner.jpg" />
10  </swiper-item>
11 </swiper>
```

在上述代码中，第 1 行代码给 swiper 组件将指示点的颜色设置为白色半透明、当前选中的指示点颜色设置为白色并显示指示点、采用衔接滑动和自动切换图片。

完成上述代码后，运行程序，轮播图的实现效果如图 4-11 所示。



图4-11 轮播图的实现效果

③ 在轮播图代码段下方编写功能按钮的页面结构，具体代码如下。

```
<view class="content-info-portal">
  <view>
    <image src="/images/04.png" />
    <text>私人 FM</text>
  </view>
  <view>
    <image src="/images/05.png" />
    <text>每日歌曲推荐</text>
  </view>
  <view>
    <image src="/images/06.png" />
    <text>云音乐新歌榜</text>
  </view>
</view>
```

在上述代码中，第 2～5 行代码定义了“私人 FM”区域；第 6～9 行代码定义了“每日歌曲推荐”区域；第 10～13 行代码定义了“云音乐新歌榜”区域。

功能按钮的实现效果如图 4-12 所示。



图4-12　功能按钮的实现效果

④ 在功能按钮代码段下方编写推荐歌曲的页面结构，具体代码如下。

```
<view class="content-info-list">
  <view class="list-title">推荐歌曲</view>
  <view class="list-inner">
    <view class="list-item">
      <image src="/images/cover.jpg" />
      <view>山水之间</view>
    </view>
    <view class="list-item">
      <image src="/images/cover.jpg" />
      <view>惊鸿一面</view>
    </view>
    <view class="list-item">
      <image src="/images/cover.jpg" />
      <view>稻香</view>
    </view>
    <view class="list-item">
      <image src="/images/cover.jpg" />
      <view>如果当时</view>
    </view>
    <view class="list-item">
      <image src="/images/cover.jpg" />
      <view>清明雨上</view>
    </view>
    <view class="list-item">
      <image src="/images/cover.jpg" />
      <view>有何不可</view>
    </view>
  </view>
</view>
```

在上述代码中，第 2 行代码定义了推荐歌曲列表的标题；第 3～28 行代码定义了推荐歌曲列表，其中第 4～7 行代码定义了推荐歌曲列表中的“山水之间”项，内部有 image 组件和 view 组件，分别用于展示推荐

歌曲的封面图片和标题，其余列表项的页面结构与之类似。

完成上述代码后，运行程序，推荐歌曲的实现效果如图4-13所示。



图4-13 推荐歌曲的实现效果

至此，“音乐推荐”标签页已实现。

5. 实现“播放器”标签页

“播放器”是内容区域中的第2个标签页，对应swiper-item组件的索引为1。该页面由3个部分组成，分别是音乐信息、专辑封面和播放进度，具体实现步骤如下。

① 在pages/index/index.js文件的data对象中定义播放列表数组playlist，具体代码如下。

```
playlist: [{
  id: 1,
  title: '祝你生日快乐',
  singer: '小丽',
  src: 'http://127.0.0.1:3000/1.mp3',
  coverImgUrl: '/images/cover.jpg'
}, {
  id: 2,
  title: '劳动最光荣',
  singer: '小朋',
  src: 'http://127.0.0.1:3000/2.mp3',
  coverImgUrl: '/images/cover.jpg'
}, {
  id: 3,
  title: '龙的传人',
  singer: '小华',
  src: 'http://127.0.0.1:3000/3.mp3',
  coverImgUrl: '/images/cover.jpg'
}, {
  id: 4,
  title: '小星星',
  singer: '小红',
  src: 'http://127.0.0.1:3000/4.mp3',
  coverImgUrl: '/images/cover.jpg'
}],
```

在上述代码中，id是每条记录的唯一标识，title为曲目标题，singer为演唱者，src为音频文件的链接地址，coverImgUrl为专辑封面图片的链接地址。

② 在data中定义一些状态属性，用于记录音乐的播放状态等信息，具体代码如下。

```
state: 'running',
playIndex: 0,
play: {
  currentTime: '00:00',
  duration: '00:00',
  percent: 0,
  title: '',
  singer: '',
  coverImgUrl: '/images/cover.jpg',
}
```

在上述代码中，第1行代码定义了属性state，保存了音乐播放状态，其默认值running表示正在播放。

如果改为 paused 则表示暂停，默认值为 running；第 2 行代码定义了属性 playIndex，表示当前播放的曲目在播放列表数组中的索引值；第 3～10 行代码定义了 play 对象，保存了当前播放曲目的信息，其中，currentTime 表示播放时长，duration 表示总时长，percent 表示播放进度，title、singer、coverImgUrl 分别表示当前播放的曲目标题、演唱者和封面。

③ 在 pages/index/index.js 文件中编写代码，实现在页面初次渲染时，自动选择播放列表中的第 1 个曲目，具体代码如下。

```
1 audioBam: null,
2 onReady: function () {
3   this.audioBam = wx.getBackgroundAudioManager()
4   // 默认选择第 1 曲
5   this.setMusic(0)
6 },
7 setMusic: function (index) {
8   // 设置当前播放的曲目，在后面的步骤中实现
9 }
```

在上述代码中，第 1 行代码定义了 audioBam 属性，属性值为 null；第 3 行代码通过调用 wx.getBackgroundAudioManager()方法获取 audioBam 对象；第 5 行代码通过 this 关键字调用页面中的 setMusic()函数，参数为 0，用于实现在页面初始渲染完成时，自动播放第 1 个曲目的效果，0 是第 1 个曲目的索引值。

④ 编写 setMusic()函数，实现设置当前播放的曲目，具体代码如下。

```
1  setMusic: function (index) {
2    var music = this.data.playlist[index]
3    this.audioBam.src = music.src
4    this.audioBam.title = music.title
5    this.setData({
6      playIndex: index,
7      'play.title': music.title,
8      'play.singer': music.singer,
9      'play.coverImgUrl': music.coverImgUrl,
10     'play.currentTime': '00:00',
11     'play.duration': '00:00',
12     'play.percent': 0,
13     state:'running'
14   })
15 },
```

在上述代码中，setMusic()函数的参数index表示播放列表数组中某项的索引值；第 2 行代码将playlist数组中第 index 项取出歌曲信息赋值给属性 music；第 3～4 行代码将 src、title 赋值给 audioBam 对象的相应属性；第 5～14 行代码调用 setData()方法对 playIndex 变量、play 对象和 state 变量进行赋值。

⑤ 在 app.json 文件中添加 requiredBackgroundModes 配置项，具体代码如下。

```
"requiredBackgroundModes": ["audio"],
```

在上述代码中，通过 requiredBackgroundModes 配置项申请需要后台运行的能力，类型为数组，在数组中添加 audio 项表示启用后台音频播放功能。

⑥ 在 pages/index/play.wxml 文件中删除原有代码，然后编写“播放器”标签页的页面结构，具体代码如下。

```
1 <view class="content-play">
2   <!-- 音乐信息 -->
3   <view class="content-play-info">
4     <text>{{ play.title }}</text>
5     <view>—— {{ play.singer }} ——</view>
6   </view>
7   <!-- 专辑封面 -->
```

```
 8    <view class="content-play-cover">
 9      <image src="{{ play.coverImgUrl }}" style="animation-play-state:{{ state }}" />
10    </view>
11    <!-- 播放进度和时间 -->
12    <view class="content-play-progress"></view>
13  </view>
```

在上述代码中，第3～6行代码定义了音乐信息区域；第8～10行代码定义了专辑封面区域，其中第9行代码中的animation-play-state属性用于控制专辑封面旋转动画的播放与暂停，当state属性的值为running时，说明音乐正在播放，此时封面开始旋转，当state属性的值为pause时，说明音乐暂停了，此时封面也暂停旋转；第12行代码定义了播放进度和时间区域，由于播放进度的控制实现起来较为复杂，将在后面的小节中进行讲解。

6. 实现播放器区域

首先编写播放器区域的整体结构，然后实现播放、暂停、下一曲的功能，其中播放和暂停功能使用同一个按钮实现，具体步骤如下。

① 在pages/index/index.wxml文件中编写页面播放器区域的代码，将当前播放的曲目信息显示在播放器中，具体代码如下。

```
 1  <view class="player">
 2    <image class="player-cover" src="{{ play.coverImgUrl }}" data-item="1" bindtap="changeItem" />
 3    <view class="player-info">
 4      <view class="player-info-title" data-item="1" bindtap="changeItem">{{ play.title }}</view>
 5      <view class="player-info-singer" data-item="1" bindtap="changeItem">{{ play.singer }}</view>
 6    </view>
 7    <view class="player-controls">
 8      <!-- 切换到播放列表 -->
 9      <image src="/images/01.png" data-item="2" bindtap="changeItem" />
10      <!-- 播放 -->
11      <image src="/images/02.png" />
12      <!-- 下一曲 -->
13      <image src="/images/03.png" />
14    </view>
15  </view>
```

在上述代码中，第2行代码展示了专辑封面图片，第4～5行代码展示了歌曲的标题和歌手；第9、11、13行代码分别定义了1个image组件，分别表示切换到播放列表、播放、下一曲按钮；第2、4、5、9行代码中绑定了tap事件，事件处理函数为changeItem()，添加了data-item自定义属性，用于实现点击切换到data-item自定义属性对应的标签页。

② 为了实现点击播放按钮播放音乐，再次点击为暂停音乐的效果，修改播放器区域中的播放区域，具体代码如下。

```
1  <!-- 播放/暂停 -->
2  <image wx:if="{{ state == 'paused' }}" src="/images/02.png" bindtap="play" />
3  <image wx:else src="/images/02stop.png" bindtap="pause" />
```

在上述代码中，第2～3行代码分别给image组件的tap事件绑定了play()、pause()事件处理函数。通过wx:if控制属性判断state属性的值，若state属性的值为paused，则显示第2行代码中的image组件；若state属性的值为running，则显示第3行代码中的image组件。

③ 在pages/index/index.js文件中编写事件处理函数play()和pause()，具体代码如下。

```
play: function () {
  this.audioBam.play()
  this.setData({
    state: 'running'
  })
},
pause: function () {
  this.audioBam.pause()
  this.setData({
    state: 'paused'
  })
},
```

在上述代码中，第 1～6 行代码定义了 play()函数，实现播放音乐的功能，其中第 2 行代码调用 audioBam 对象的 play()函数来实现音乐播放功能，第 3～5 行代码调用了 setData()方法，设置 state 属性的值为 running；第 7～12 行代码定义了 pause()函数，实现暂停音乐的功能，其中第 8 行代码调用 audioBam 对象的 pause()函数来实现音乐的暂停功能，第 9～11 行代码调用了 setData()方法，设置 state 属性的值为 paused。

④ 修改“下一曲”图片按钮的代码，具体代码如下。

```
<image src="/images/03.png" bindtap="next" />
```

在上述代码中，给 image 组件的 tap 事件绑定了 next()事件处理函数。

⑤ 编写事件处理函数 next()函数，实现点击播放下一曲的操作，具体代码如下。

```
next: function () {
  var index = this.data.playIndex >= this.data.playlist.length - 1 ? 0 : this.data.playIndex + 1
  this.setMusic(index)
},
```

在上述代码中，第 2 行代码通过三元表达式判断当前正在播放曲目的索引值是否大于等于播放列表的长度，若为 true，则 index 值为 0；若为 false，则将当前正在播放曲目的索引值+1；第 3 行代码调用 setMusic()方法实现音频的播放。

7. 实现播放进度的控制

在“播放器”标签页的底部有播放进度和时间区域，用于显示播放进度，实现该功能具体步骤如下。

① 在 pages/index/play.wxml 文件中编写播放进度和时间区域的页面结构，具体代码如下。

```
<view class="content-play-progress">
  <text>{{ play.currentTime }}</text>
  <view>
    <slider bindchanging="sliderChanging" bindchange="sliderChange" activeColor="#d33a31" block-size="12" backgroundColor="#dadada" value="{{ play.percent }}" />
  </view>
  <text>{{ play.duration }}</text>
</view>
```

在上述代码中，通过 slider 组件定义进度条，其值为 play.percent；在 slider 组件上绑定了 changing 和 change 事件，事件处理函数分别为 sliderChanging()和 sliderChange()。

② 在 pages/index/index.js 文件中控制进度条的进度和时间的显示，具体代码如下。

```
onReady: function () {
  this.audioBam = wx.getBackgroundAudioManager()
  // 默认选择第 1 曲
```

```
4   this.setMusic(0)
5   // 播放失败检测
6   this.audioBam.onError(() => {
7     console.log('播放失败：' + this.audioBam.src)
8   })
9   // 播放完成自动换下一曲，监听音频自然播放结束的事件
10  this.audioBam.onEnded(() => {
11    this.next()
12  })
13  // 监听音频播放进度更新事件，获取音乐状态信息
14  var updateTime = 0  // 记录上次更新的时间，用于限制1秒只能更新1次进度
15  this.audioBam.onTimeUpdate(() => {
16    var currentTime = parseInt(this.audioBam.currentTime)
17    if (!this.sliderChangeLock && currentTime !== updateTime) {
18      updateTime = currentTime
19      this.setData({
20        'play.duration': formatTime(this.audioBam.duration || 0),
21        'play.currentTime': formatTime(currentTime),
22        'play.percent': currentTime / this.audioBam.duration * 100
23      })
24    }
25  })
26 },
```

在上述代码中，第6～8行代码通过调用audioBam对象的onError()方法监听音乐播放错误事件；第10～12行代码通过调用audioBam对象的onEnded()方法，实现音频自然播放结束之后播放下一首歌曲；第15～25行代码通过调用audioBam对象的onTimeUpdate()方法，获取音乐状态信息，更新播放进度，其中第17行代码中的this.sliderChangeLock表示用户当前是否正在操作slider组件，如果正在操作，则阻止自动更新播放进度；第20行和第21行代码调用了formatTime()函数，用于将秒数转换为“分:秒”时间格式，该函数将在下一步中编写。

③ 在Page()函数前定义一个formatTime()函数，具体代码如下。

```
1  // 格式化时间
2  function formatTime(time) {
3    var minute = Math.floor(time / 60) % 60;
4    var second = Math.floor(time) % 60
5    return (minute < 10 ? '0' + minute : minute) + ':' + (second < 10 ? '0' + second : second)
6  }
```

④ 在Page({})中编写sliderChangeLock属性、sliderChanging()函数和sliderChange()函数，具体代码如下。

```
1  sliderChangeLock: false,
2  sliderChanging: function (e) {
3    var second = e.detail.value * this.audioBam.duration / 100
4    this.sliderChangeLock = true
5    this.setData({
6      'play.currentTime': formatTime(second),
7    })
8  },
9  sliderChange: function (e) {
10   var second = e.detail.value * this.audioBam.duration / 100
11   this.audioBam.seek(second)
12   setTimeout(() => {
13     this.sliderChangeLock = false
14   }, 1000)
15 },
```

在上述代码中，第 2～8 行代码用于在用户操作 slider 组件时阻止播放进度自动更新，并显示用户选择的时间；第 3 行代码和第 10 行代码用于获取用户选择的时间；第 11 行代码用于调节音乐的播放进度；第 12～14 行代码用于在延迟 1 秒后允许播放进度自动更新。

至此，播放进度的控制已经实现。

8. 实现“播放列表”标签页

“播放列表”是内容区域中的第 3 个标签页，对应 swiper-item 组件的索引为 2，该页面显示当前播放的曲目列表，具体实现步骤如下。

① 在 pages/index/playlist.wxml 文件中编写“播放列表”标签页的页面结构，具体代码如下。

```
1 <scroll-view class="content-playlist" scroll-y>
2   <view class="playlist-item" wx:for="{{ playlist }}" wx:key="id" bindtap="change"
data-index="{{ index }}">
3     <image class="playlist-cover" src="{{ item.coverImgUrl }}" />
4     <view class="playlist-info">
5       <view class="playlist-info-title">{{ item.title }}</view>
6       <view class="playlist-info-singer">{{ item.singer }}</view>
7     </view>
8     <view class="playlist-controls">
9       <text wx:if="{{ index == playIndex }}">正在播放</text>
10    </view>
11  </view>
12 </scroll-view>
```

在上述代码中，第 1～12 行代码定义了 scroll-view 组件，通过设置 scroll-y 属性实现播放列表的纵向滚动，其中第 2～11 行代码通过列表渲染将 playlist 数组渲染到页面上，给 view 组件绑定 tap 事件，事件处理函数为 change()。第 9 行代码判断当前索引是否与当前播放的曲目在播放列表数组中的索引相同，如果相同说明该曲目是正在播放的状态，该 text 组件显示。

② 在 pages/index/index.js 文件中编写 change()函数，实现点击播放列表中的某一项时进行该曲目的播放，具体代码如下。

```
1 change: function (e) {
2   this.setMusic(e.currentTarget.dataset.index)
3 }
```

在上述代码中，当用户点击播放列表中的某一项后会触发 change()函数，实现对应曲目的播放。

完成上述代码后，运行程序，“音乐播放器”微信小程序的实现效果如图 4-1～图 4-3 所示。

至此，“音乐播放器”微信小程序已开发完成。

【案例 4-2】录音机

录音机是生活中的常用工具，它可以在开会的时候记录说话人的声音，也可以在生活中留下一段美妙歌声。录音机可以记录声音和播放声音，因此其成为新闻工作者工作的重要器材。本案例将对“录音机”微信小程序的开发进行详细讲解。

案例分析

“录音机”微信小程序的页面效果如图 4-14 所示。



图4–14 “录音机”微信小程序的页面效果

在图 4–14 中，“录音机”微信小程序页面分为顶部区域和按钮控制区域。其中，顶部区域展示录音时长，按钮控制区域中从左到右的 3 个按钮分别是“播放录音”按钮、“开始/暂停录音”按钮和“停止录音”按钮，这 3 个按钮分别实现了播放录音、开始或暂停录音、停止录音的功能。

案例实现

1. 准备工作

在开发本案例前，需要先完成一些准备工作，主要包括创建项目、配置导航栏和复制素材，具体步骤如下。

① 创建项目。在微信开发者工具中创建一个新的微信小程序项目，项目名称为“录音机”，模板选择“不使用模板”。

② 配置导航栏。在 pages/index/index.json 文件中配置页面导航栏，具体代码如下。

```
{
  "navigationBarTitleText": "录音机"
}
```

上述代码将导航栏标题设置为“录音机”。“录音机”导航栏的效果如图 4–15 所示。

③ 复制素材。从本书配套资源中找到本案例，复制以下素材到本项目中。

- pages/index/index.wxss 文件，该文件中保存了本项目的页面样式素材。
- utils 文件夹，该文件夹保存了本项目所用的公共模块素材。

上述步骤操作完成后，“录音机”微信小程序的目录结构如图 4–16 所示。

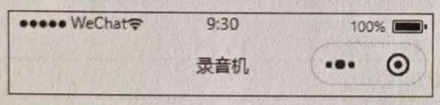


图4–15 “录音机”导航栏的效果

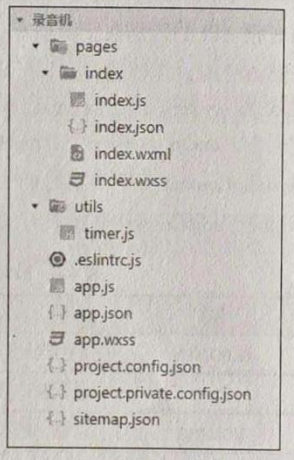


图4–16 “录音机”微信小程序的目录结构

至此，准备工作已经全部完成。

2. 初始化录音功能

在pages/index/index.js 文件的Page( )函数前编写获取RecorderManager实例并监听录音结束事件的代码，具体如下。

```
var rec = wx.getRecorderManager()
var tempFilePath = null
rec.onStop(res => {
  tempFilePath = res.tempFilePath
  console.log('录音成功：' + tempFilePath)
})
```

在上述代码中，第 1 行代码调用了 wx.getRecorderManager( )方法获取 RecorderManager 实例；第 2 行代码定义了 tempFilePath 变量，用于保存录音文件的临时路径；第 3～6 行代码调用了 rec 对象的 onStop( )方法，用于监听录音结束事件，在录音停止时，将录音文件的路径保存在 tempFilePath 变量中。

在 Page({ })中定义页面中所需的数据，具体代码如下。

```
data: {
  time: '00:00:00',    // 录音时长
  state: 0,            // 录音状态，0 表示停止，1 表示开始，2 表示暂停
}
```

在上述代码中，第 2 行代码定义了 time 属性，属性值为 00:00:00，表示初始录音时长为 00:00:00；第 3 行代码定义了 state 属性，用于保存录音状态，其值为 0 表示停止，值为 1 表示开始，值为 2 表示暂停，默认值为 0。

3. 实现“录音机”微信小程序的页面结构

在 pages/index/index.wxml 文件中编写页面结构，具体代码如下。

```
<view class="top">
  <view class="top-title">录音机</view>
  <view class="top-time">{{ time }}</view>
</view>
<view class="control">
  <view class="btn btn-play" bindtap="play" hover-class="btn-hover" hover-stay-time="50"></view>
  <view class="btn btn-rec {{ state === 1 ? 'btn-rec-pause' : 'btn-rec-normal' }}" bindtap="rec" hover-class="btn-hover" hover-stay-time="50"></view>
  <view class="btn btn-stop" bindtap="stop" hover-class="btn-hover" hover-stay-time="50"></view>
</view>
```

在上述代码中，第 1～4 行代码定义了顶部区域，其中第 3 行代码定义了 view 组件展示了录制时长。第 5～9 行代码定义了按钮控制区域，其中，第 6 行代码表示“播放录音”按钮，为 view 组件绑定 tap 事件，事件处理函数为 play( )；第 7 行代码表示“开始/暂停录音”按钮，为 view 组件绑定 tap 事件，事件处理函数为 rec( )，且通过三元表达式判断 state 属性的值是否等于 1，若为 true，说明此时正在录音，则为用户提供“暂停”按钮，若为 false，说明此时处于停止或暂停状态，则为用户提供“开始”按钮；第 8 行代码表示“停止录音”按钮，为 view 组件绑定 tap 事件，事件处理函数为 stop( )。

至此，“录音机”微信小程序的页面结构已经实现。

4. 实现录音功能

“录音机”微信小程序为用户提供了开始或暂停录音、停止录音、播放录音的功能。因为播放录音功能需要提供音频，所以先完成开始或暂停、停止录音功能的编写。接下来将分别实现这些功能，具体步骤如下。

① 在 pages/index/index.js 文件的开头位置引入 timer 模块，用于对录音时间进行计时，具体代码如下。

```
const timer = require('../../utils/timer.js')
```

在上述代码中，timer是一个对象，该对象的start()方法用于开始计时；pause()方法用于暂停计时；reset()方法用于停止计时并复位。该对象有一个onTimeUpdate事件，该事件会在计时开始后每秒自动触发一次。

② 在Page({})中编写rec()函数，实现开始或暂停录音，具体代码如下。

```
1  rec: function () {
2    switch (this.data.state) {
3      case 0:
4        rec.start()
5        timer.onTimeUpdate(time => {
6          this.setData({ time })
7        })
8        timer.start()
9        this.setData({ time: '00:00:00', state: 1 })
10       break
11     case 1:
12       rec.pause()
13       timer.pause()
14       this.setData({ state: 2 })
15       break
16     case 2:
17       rec.resume()
18       timer.start()
19       this.setData({ state: 1 })
20       break
21   }
22 },
```

在上述代码中，第2～21行代码通过switch语句实现根据不同条件执行不同的代码。第3～10行代码为第1个条件，state属性值为0，表示此时录音处于未开始或停止状态，执行开始录音的操作，其中，第5～7行代码设置了onTimeUpdate()事件处理函数，用于实现当时间更新后将时间更新到页面中；第11～15行代码为第2个条件，state属性值为1，表示此时录音处于开始状态，执行暂停录音的操作；第16～20行代码为第3个条件，state属性值为2，表示此时录音处于暂停状态，执行继续录音的操作。

③ 在Page({})中编写stop()函数，实现停止录音，具体代码如下。

```
1  stop: function () {
2    rec.stop()
3    timer.reset()
4    this.setData({ state: 0 })
5  },
```

在上述代码中，第2行代码调用了rec对象的stop()方法，用于停止录音；第3行代码调用了timer对象的reset()方法，用于重置录音时间；第4行代码调用了setData()方法，将state设置为0，表示当前状态为停止录制。

5. 实现播放录音功能

在按钮控制区域中，实现点击“播放录音”按钮播放录音，具体步骤如下。

① 在pages/idnex/index.js文件的开头位置获取InnerAudioContext实例，具体代码如下。

```
var audioCtx = wx.createInnerAudioContext()
```

在上述代码中，调用wx.createInnerAudioContext()方法获取InnerAudioContext实例，用于实现音频播放功能。

② 在Page({})中编写play()函数，实现播放录音，具体代码如下。

```
1  play: function () {
2    if (this.data.state > 0) {
3      // 第1种情况，录音尚未完成（在后面的步骤中实现）
4    } else if (tempFilePath) {
```

```
    // 第2种情况，录音已完成
    audioCtx.src = tempFilePath
    audioCtx.play()
    this.setData({ time: '播放录音' })
  } else {
    // 第3种情况，尚未录音
    this.setData({ time: '暂无录音' })
  }
}
```

在上述代码中，第2～12行代码进行了条件判断，若this.data.state > 0，表示处于开始录制或者暂停录制状态，录音尚未完成，此时先不做任何处理；若属性tempFilePath属性值为true，表示录音已完成，则将audioCtx对象的src设置为tempFilePath，调用play()方法实现录音的播放；否则为尚未录音状态，调用setData()方法将time设置为“暂无录音”。

③ 当录音处于开始录制或者暂停录制时，应该先自动停止录音，并在停止录音后，执行播放录音的代码。找到rec.onStop()代码进行修改，修改后的代码如下。

```
var onStopCallBack = null
rec.onStop(res => {
  tempFilePath = res.tempFilePath
  console.log('录音成功：' + tempFilePath)
  onStopCallBack && onStopCallBack(tempFilePath)
})
```

在上述代码中，第1行和第5行代码为新增代码。第1行代码定义了变量onStopCallBack，该变量的值为null；第2～6行代码为监听录音结束之后的事件，在录音结束之后获取录音文件路径，并调用了onStopCallBack()函数，该函数用于播放录音。onStopCallBack()函数将在接下来的步骤中实现。

④ 在play()函数中编写第1种情况的代码，具体代码如下。

```
// 第1种情况，录音尚未完成
onStopCallBack = tempFilePath => {
  onStopCallBack = null
  audioCtx.src = tempFilePath
  audioCtx.play()
  this.setData({ time: '播放录音' })
}
this.stop()
```

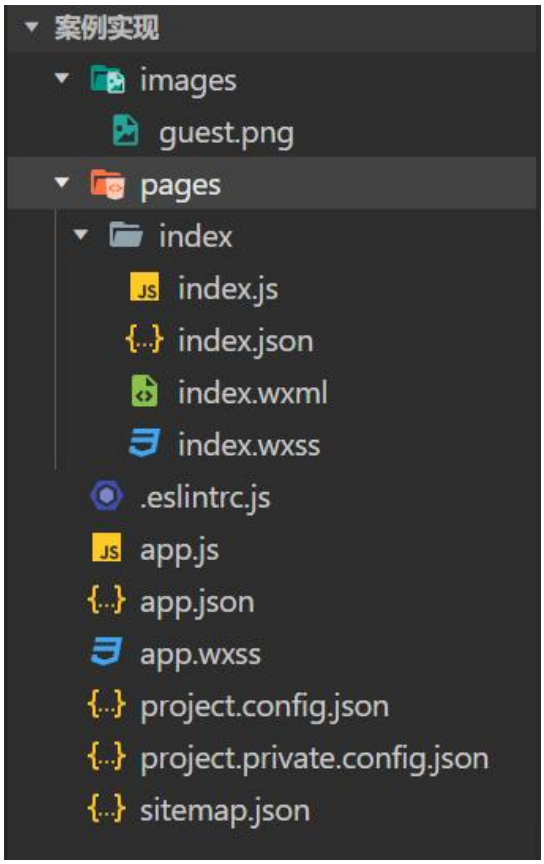
在上述代码中，第2～7行代码将onStopCallBack变量赋值为函数，其中，第3行代码将onStopCallBack变量赋值为null，这样可以避免每次停止录音时都调用onStopCallBack()函数；第4行代码将录音文件的临时路径保存在audioCtx对象的src属性中；第5行代码调用play()方法播放录音；第6行代码将time赋值为播放录音。第8行代码调用stop()方法停止录音。录音停止后会执行onStop()事件处理函数中的代码。

完成上述代码后，运行程序，“录音机”微信小程序的实现效果如图4-14所示。

至此，“录音机”微信小程序已开发完成。

【案例4-3】头像上传下载

头像上传下载是微信小程序开发中常见的一个功能，一般会出现在用户中心模块中，用于设置用户的头像。为了方便读者学习相关技术，下面将会开发一个专门用于实现头像上传下载的微信小程序。





案例实现

1. 准备工作

在开发本案例前，需要先完成一些准备工作，主要包括创建项目、配置导航栏、复制素材和启动服务器，具体步骤如下。

① 创建项目。在微信开发者工具中创建一个新的微信小程序项目，项目名称为“头像上传下载”，模板选择“不使用模板”。

② 配置导航栏。在 pages/index/index.json 文件中配置页面导航栏，具体代码如下。

```
{
  "navigationBarTitleText": "头像上传下载"
}
```

上述代码将导航栏标题设置为“头像上传下载”。“头像上传下载”导航栏的效果如图 4-19 所示。

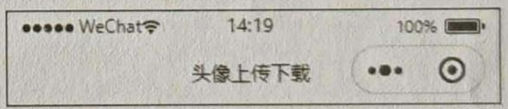


图4-19 “头像上传下载”导航栏的效果

③ 复制素材。从本书配套资源中找到本案例，复制以下素材到本项目中。

- pages/index/index.wxss 文件，该文件中保存了本项

目的页面样式素材。

- images 文件夹，该文件夹保存了本项目所用图片的素材。

④ 启动服务器。从本书配套资源中找到本案例的源代码，进入“服务器端”文件夹，该文件夹下的内容为 Node.js 本地 HTTP 服务器程序。打开命令提示符，切换工作目录到当前目录，然后在命令提示符中执行如下命令，启动服务器。

```
node index.js
```

上述步骤操作完成后，“头像上传下载”微信小程序的目录结构如图 4-20 所示。

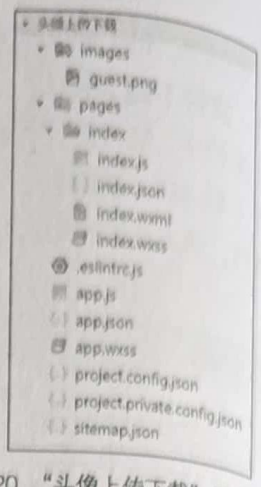


图4-20 “头像上传下载”微信小程序的目录结构

至此，准备工作已经全部完成。

2. 实现“头像上传下载”微信小程序的页面结构

在 pages/index/index.wxml 文件中编写页面结构，具体代码如下。

```
1 <view class="imgbox">
2   <image src="{{ imgUrl }}" mode="aspectFit" />
3   <button type="primary" size="mini" bindtap="changeImg">更改头像</button>
4   <button type="primary" size="mini" bindtap="upload">头像上传</button>
5   <button type="primary" size="mini" bindtap="download">头像下载</button>
6 </view>
```

在上述代码中，第 3 行代码为“更改头像”按钮绑定了 changeImg()函数，当用户点击“更改头像”按钮时会触发 changeImg()事件处理函数；第 4 行代码为“头像上传”按钮绑定了 upload()函数；第 5 行代码为“头像下载”按钮绑定了 download()函数。

至此，“头像上传下载”微信小程序的页面结构已经实现。

3. 实现“头像上传下载”微信小程序的页面逻辑

在 pages/index/index.js 文件的 Page({})中编写逻辑代码，具体步骤如下。

① 在 data 中定义初始数据，具体代码如下。

```
1 data: {
2   imgUrl: '/images/guest.png',
3   tempFilePath: null
4 },
5 uploadFileUrl: null,
```

在上述代码中，第 2 行代码中的 imgUrl 为头像的初始显示图片；第 3 行代码定义了 tempFilePath 属性，属性值为 null，用于保存图片文件临时路径；第 5 行代码定义了 uploadFileUrl 属性，用于保存图片上传之后的图片路径。

② 编写事件处理函数 changeImg()，实现图片选择，具体代码如下。

```
1  changeImg: function () {
2    wx.chooseMedia({
3      count: 1,
4      mediaType: ['image'],
5      sourceType: ['album', 'camera'],
6      success: res => {
7        var tempFilePath = res.tempFiles[0].tempFilePath
8        this.setData({
9          tempFilePath: tempFilePath,
10         imgUrl: tempFilePath
11       })
12     }
```

```
13  })
14 },
```

在上述代码中，第 2～13 行代码通过 wx.chooseMedia()方法进行图片选择。其中，第 3 行代码定义了 count 属性，属性值为 1，表示最多可以选择的文件个数为 1；第 4 行代码定义了 mediaType 属性，属性值为 image，表示文件类型只能是拍摄的图片或从相册选择的图片；第 5 行代码定义了 sourceType 属性，属性值为 album 和 camera，表示图片的选择来源为从相册选择和使用相机拍摄；第 6～12 行代码定义了接口调用成功的回调函数 success()，将文件路径保存在 tempFilePath 属性和 imgUrl 属性中。

③ 编写事件处理函数 upload()，实现头像的上传，具体代码如下。

```
1  upload: function () {
2    // 如果没有更改照片提示更改后再上传
3    if (!this.data.tempFilePath) {
4      wx.showToast({
5        title: '请您更改头像之后再进行上传操作',
6        icon: 'none',
7        duration: 2000
8      })
9      return
10   }
11   // 确认更改头像之后再上传
12   wx.uploadFile({
13     filePath: this.data.tempFilePath,
14     name: 'image',
15     url: 'http://localhost:3000/upload',
16     success: res => {
17       this.uploadFileUrl = JSON.parse(res.data).file
18       console.log('上传成功')
19     }
20   })
21 },
```

在上述代码中，第 3～9 行代码通过 if 判断是否更改头像，当图片文件临时路径 tempFilePath 为 false 时，表示没有更改头像，通过 wx.showToast()方法弹出消息提示框，提示“请您更改头像之后再进行上传操作”；第 12～20 行代码通过 wx.uploadFile()方法将本地资源上传到开发者服务器，其中第 15 行代码为服务器地址，第 17 行代码将服务器返回的图片地址保存到 uploadFileUrl 属性中，用于后续开发下载图片的功能。

④ 在微信开发者工具的本地设置中勾选“不校验合法域名、web-view（业务域名）、TLS 版本以及 HTTPS 证书”复选框。

⑤ 编写事件处理函数 download()，实现图片的下载，具体代码如下。

```
1  download: function () {
2    if (!this.uploadFileUrl) {
3      wx.showToast({
4        title: '请您上传头像之后再进行下载操作',
5        icon: 'none',
6        duration: 2000
7      })
8      return
9    }
10   wx.showLoading({
11     title: '图片下载中，请稍后……',
12   })
13   wx.downloadFile({
```

```
14    url: this.uploadFileUrl,
15    success: res => {
16      wx.hideLoading()
17      console.log('下载完成')
18      wx.previewImage({
19        urls: [ res.tempFilePath ]
20      })
21    }
22  })
23 }
```

在上述代码中，第2～9行代码用于判断上传的图片路径是否存在，若!this.uploadFileUrl为true，则通过wx.showToast()方法弹出消息提示框，提示“请您上传头像之后再进行下载操作”；第10～12行代码用于提示用户图片正在下载；第13～22行代码通过wx.downloadFile()方法将图片下载到本地，其中第14行代码设置了url地址、第18～20行代码用于在图片下载完成后通过wx.previewImage()预览图片。

完成上述代码后，运行程序，“头像上传下载”微信小程序的实现效果如图4-17所示。

至此，“头像上传下载”微信小程序已经开发完成。

【案例4-4】模拟时钟

“模拟时钟”微信小程序是一个简约风格的动态时钟，该时钟时间与系统时间一致，且时针、分针、秒针会与系统时间同步更新，用户可以很方便地查看时间。下面将对“模拟时钟”微信小程序进行详细讲解。

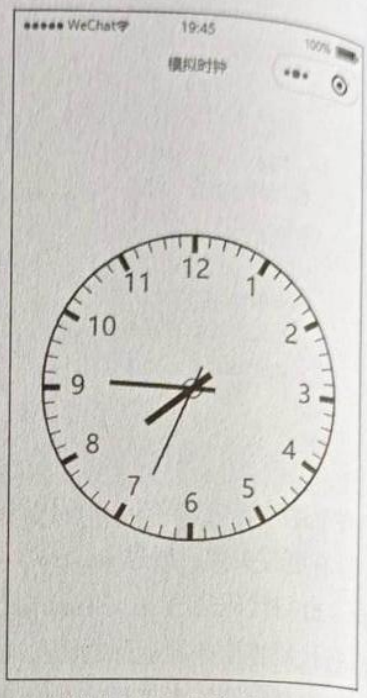


图4-21 “模拟时钟”微信小程序的页面效果

案例分析

“模拟时钟”微信小程序利用canvas组件绘制时钟，刻度为12个刻度，需要分别画出中心圆、外层大圆、时针、分针、秒针。“模拟时钟”微信小程序的页面效果如图4-21所示。

知识储备

1. canvas组件

在HTML中，<canvas>标签可用于图形的绘制，也可用于创建图片特效和动画。在微信小程序中，canvas组件也起着类似作用，可用于自定义绘制图形，该组件支持2D和WebGL的绘图。

canvas组件通过<canvas>标签来定义，示例代码如下。

```
<canvas></canvas>
```

canvas组件的常用属性如表4-11所示。

表4-11 canvas组件的常用属性

属性	类型	说明
type	string	指定canvas组件的类型，支持2D和WebGL
canvas-id	string	canvas组件的唯一标识符，若指定了type属性则无须再指定该属性
disable-scroll	boolean	当在canvas组件中移动时且有绑定手势事件时，禁止屏幕滚动及下拉刷新，默认值为false

① 创建项目。在微信开发者工具中创建一个新的微信小程序项目，项目名称为“模拟时钟”，模板选择“不使用模板”。

② 配置导航栏。在 pages/index/index.json 文件中配置页面导航栏，具体代码如下。

```
{
  "navigationBarTitleText": "模拟时钟"
}
```

上述代码将导航栏标题设置为“模拟时钟”。“模拟时钟”导航栏的效果如图 4-25 所示。

上述步骤操作完成后，“模拟时钟”微信小程序的目录结构如图 4-26 所示。

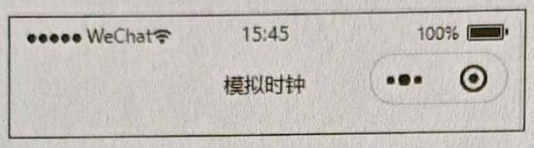


图4-25 “模拟时钟”导航栏的效果

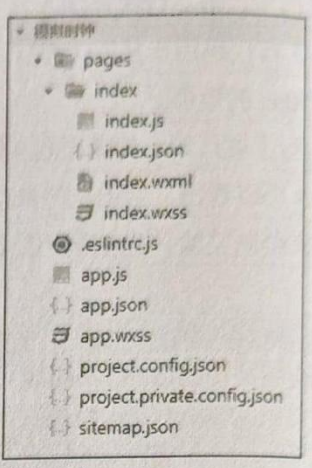


图4-26 “模拟时钟”微信小程序的目录结构

至此，准备工作已经全部完成。

2. 初始化画布

在绘制模拟时钟之前，需要创建画布并进行一些初始化操作，具体如下。

① 在 pages/index/index.wxml 文件中定义 canvas 组件，具体代码如下。

```
<canvas id="myCanvas" type="2d"></canvas>
```

上述代码用 canvas 组件创建画布。其中，type 属性值为 2d，表示 Canvas 2D 接口。

② 在 pages/index/index.wxss 文件中编写 canvas 组件的样式，具体代码如下。

```
1 #myCanvas {
2   width: 100%;
3   height: 100%;
4   position: fixed;
5 }
```

上述代码设置了 canvas 组件的宽、高为 100%，占满整个屏幕，并且设置固定定位。

③ 在项目根目录下创建 utils 文件夹，将绘制功能封装到 utils/drawClock.js 文件中，该文件的基本代码如下。

```
1 module.exports = canvas => {
2   const ctx = canvas.getContext('2d')
3   // 计算表盘半径，留出 30px 外边距
4   var radius = canvas.width / 2 - 30
5   return () => {
6     // 在此处编写绘制代码
7   }
8 }
```

上述代码通过 module.exports 将页面中定义的方法暴露出去。其中，第 2 行代码用于获取 RenderingContext 实例；第 4 行代码用于计算表盘的半径，为了美观，预留 30px 的宽度。

④ 在 pages/index/index.js 文件中引入模块，具体代码如下。

```
const drawClock = require('../../utils/drawClock.js')
```

⑤ 在 pages/index/index.js 文件中编写 onReady()函数，实现页面初次渲染时获取 Canvas 实例，具体代码如下。

```
1  onReady: function () {
2    wx.createSelectorQuery()
3    .select('#myCanvas')
4    .fields({ node: true, size: true })
5    .exec(res => {
6      const canvas = res[0].node
7      canvas.width = res[0].width
8      canvas.height = res[0].height
9      const draw = drawClock(canvas)
10     draw()
11   })
12 }
```

在上述代码中，第 9～10 行代码调用模块 drawClock 中的函数，在页面初次渲染的时候进行模拟时钟的绘制。

⑥ 在 utils/drawClock.js 文件中编写绘制代码，具体代码如下。

```
1  return () => {
2    // 设置坐标轴原点为画布的中心点
3    ctx.translate(canvas.width / 2, canvas.height / 2)
4    // 绘制表盘
5    drawDial(ctx, radius)
6    // 绘制指针
7    drawHand(ctx, radius)
8  }
```

在上述代码中，第 3 行代码通过调用 ctx 对象的 translate()方法设置坐标轴原点为中心点；第 5 行代码调用 drawDial()函数绘制表盘；第 7 行代码调用 drawHand()函数绘制指针。绘制表盘和绘制指针的函数在后续的步骤中实现。

3. 绘制表盘

在 utils/drawClock.js 文件中编写表盘部分，具体步骤如下。

① 编写表盘整体部分，包括外层大圆和中心圆部分，具体代码如下。

```
1  function drawDial(ctx, radius) {
2    // 绘制外层大圆
3    ctx.lineWidth = '2'                                    // 设置线条宽度为 2px
4    ctx.beginPath()                                        // 开始一条路径
5    ctx.arc(0, 0, radius, 0, 2 * Math.PI, true)            // 画弧线
6    ctx.stroke()                                           // 绘制
7    // 绘制中心圆
8    ctx.lineWidth = '1'
9    ctx.beginPath()
10   ctx.arc(0, 0, 8, 0, 2 * Math.PI, true)                 // 中心圆半径为 8px
11   ctx.stroke()
12 }
```

在上述代码中，绘制外层大圆和中心圆方法类似，第 3～6 行代码用于绘制外层大圆，第 8～11 行代码用于绘制中心圆。

② 将常用的角度换算成弧度。在时钟中，有大刻度盘和小刻度盘两种，大刻度盘被分为 12 小格，1 格代表 5 分钟，每一格所对角的度数为 30°，小刻度盘被分为 60 小格，1 格代表 1 分钟，每一格所对角的度数

为6°。

由于ctx.rotate()方法的参数是弧度不是角度，可以将常用的角度换算成弧度以便于使用，在utils/drawClock.js文件开头位置添加如下代码。

```
// 将角度转换为弧度
const D6 = 6 * Math.PI / 180
const D30 = 30 * Math.PI / 180
const D90 = 90 * Math.PI / 180
```

在上述代码中，D6、D30和D90表示角度分别为6°、30°和90°，但它们的值已经换算为弧度。

③ 编写代码，绘制大刻度盘和小刻度盘，具体代码如下。

```
function drawDial(ctx, radius) {
  原有代码……
  // 绘制大刻度盘
  ctx.lineWidth = '5'
  // 从三点钟开始，转圈进行绘制
  for (var i = 0; i < 12; ++i) {
    // 以原点为中心顺时针旋转，多次调用旋转的角度会叠加，从而画出倾斜的线
    ctx.rotate(D30)                    // 大刻度盘绘制12个线条
    ctx.beginPath()
    // 设置起始点，现在原点是在中心点，调用moveTo()方法将线条移动到外层大圆上
    ctx.moveTo(radius, 0)
    // 设置终点
    ctx.lineTo(radius - 15, 0)         // 大刻度长度15px
    ctx.stroke()
  }
  // 绘制小刻度盘
  ctx.lineWidth = '1'
  for (var i = 0; i < 60; ++i) {
    ctx.rotate(D6)
    ctx.beginPath()
    ctx.moveTo(radius, 0)
    ctx.lineTo(radius - 10, 0)         // 小刻度长度10px
    ctx.stroke()
  }
}
```

在上述代码中，绘制大刻度盘和小刻度盘方法类似，第4～15行代码用于绘制大刻度盘，第17～24行代码用于绘制小刻度盘。第6行代码中，通过循环绘制大刻度盘刻度，大刻度盘刻度共有12条；第18行代码中，通过循环绘制小刻度盘刻度，小刻度盘刻度共有60条。

④ 绘制表盘上的数字，具体代码如下。

```
function drawDial(ctx, radius) {
  原有代码……
  // 绘制数字
  ctx.font = '22px sans-serif'
  ctx.textBaseline = 'middle'          // 文本垂直居中
  // 文本距离时钟中心点半径，让文字与表盘线有距离
  var r = radius - 30
  // 文本位置是绕外圈圆的，所以要计算文本坐标
  for (var i = 1; i <= 12; ++i) {
    // 利用三角函数计算文本坐标
    var x = r * Math.cos(D30 * i - D90)
    var y = r * Math.sin(D30 * i - D90)
    // 位置进行调整
```

```
    if (i > 10) {
      // 在画布上绘制文本，fillText(文本，左上角x坐标，左上角y坐标)
      ctx.fillText(i, x - 12, y)    // 绘制11和12
    } else {
      ctx.fillText(i, x - 6, y)     // 绘制1~10
    }
  }
}
```

在上述代码中，第11～12行代码用于计算绘制数字的x、y坐标，其中，“D30 * i”用于计算每个数字旋转的度数。假设x、y为0，fillText()方法会把数字绘制到水平方向，也就是3点钟的位置上。当i的值为1时，则会把数字1绘制到4点钟的位置上。为了将数字1绘制到1点钟的位置上，需要将D30 * i的计算结果减D90（90°）。第16行和第18行代码用于调用fillText()方法将数字作为文本绘制到表盘上，其中，将x坐标减12或减6是为了将数字与刻度线对齐。

4. 绘制指针

在utils/drawClock.js文件中编写绘制指针的代码，具体代码如下。

```
function drawHand(ctx, radius) {
  var t = new Date()                  // 获取当前时间
  var h = t.getHours()                // 小时
  var m = t.getMinutes()              // 分
  var s = t.getSeconds()              // 秒
  h = h > 12 ? h - 12 : h             // 将24小时制转换为12小时制
  // 时间从3点开始，逆时针旋转90°，指向12点
  ctx.rotate(-D90)
  // 绘制时针
  ctx.save()                          // 记录旋转状态
  ctx.rotate(D30 * (h + m / 60 + s / 3600))
  ctx.lineWidth = '6'
  ctx.beginPath()
  ctx.moveTo(-20, 0)                  // 线条起点（针尾留出20px）
  ctx.lineTo(radius / 2.6, 0)         // 线条长度
  ctx.stroke()
  ctx.restore()                       // 恢复旋转状态，避免旋转叠加
  // 绘制分针
  ctx.save()
  ctx.rotate(D6 * (m + s / 60))
  ctx.lineWidth = '4'
  ctx.beginPath()
  ctx.moveTo(-20, 0)
  ctx.lineTo(radius / 1.8, 0)
  ctx.stroke()
  ctx.restore()
  // 绘制秒针
  ctx.save()
  ctx.rotate(D6 * s)
  ctx.lineWidth = '2'
  ctx.beginPath()
  ctx.moveTo(-20, 0)
  ctx.lineTo(radius / 1.6, 0)
  ctx.stroke()
  ctx.restore()
}
```

在上述代码中，第2行代码为获取当前时间，将时间保存在 t 中；第3～5行代码获取当前的时、分、秒；第6行代码将24小时制转换为12小时制；第10行代码调用 save()方法保存绘图上下文；第11行代码通过调用 rotate()方法将指针进行旋转；第12行代码设置时针线条宽度为6px；第13行代码设置开始路径；第14行代码通过调用 moveTo()方法设置针尾，时针以针尾起点画线；第15行代码通过调用 lineTo()方法计算时针长度；第16行代码进行画线；第17行代码用于恢复之前保存的绘图上下文。

5. 实现时钟走动效果

上述步骤完成后，实现了静态的模拟时钟，接下来让时钟动起来，具体步骤如下。

① 在 Page({})中定义一个 timer 属性，用于保存定时器，具体代码如下。

```
timer: null, // 定时器
```

② 修改 onReady()函数中 exec()方法的代码，在调用 draw()函数后设置定时器，实现每秒调用一次 draw()函数，具体代码如下。

```
1 .exec(res => {
2   原有代码……
3   this.timer = setInterval(draw, 1000)
4 })
```

③ 编写 onUnload()函数，在页面卸载时清除定时器，具体代码如下。

```
1 onUnload: function () {
2   clearInterval(this.timer)
3 }
```

上述代码通过调用 clearInterval()方法来清除定时器。

④ 修改 utils/clock.js 文件中的 return 代码，在绘制时钟前先清除画布，并在绘制完成后将画布恢复成初始状态，方便下次绘制，具体代码如下。

```
1 return () => {
2   // 清除画布
3   ctx.clearRect(0, 0, canvas.width, canvas.height);
4   原有代码……
5   // 绘制完成后将画布恢复成初始状态
6   ctx.rotate(D90)
7   ctx.translate(-canvas.width / 2, -canvas.height / 2)
8   ctx.restore()
9 }
```

在上述代码中，第2～3行和第5～8行代码为新增代码。其中，第3行代码通过调用 clearRect()方法来清除画布；第6行代码调用 rotate()方法将画布恢复成初始旋转角度；第7行代码调用 translate()方法重新设置原点；第8行代码调用 restore()方法恢复之前保存的绘图上下文。

完成上述代码后，运行程序，“模拟时钟”微信小程序的实现效果如图4-21所示。

至此，“模拟时钟”微信小程序已开发完成。

本章小结

本章主要讲解了微信小程序的常用 API，并结合案例演示了这些 API 在微信小程序中的应用。通过本章的学习，读者应掌握本章所讲的常用 API 的使用，能够利用背景音频 API、录音 API、音频 API、选择媒体 API、图片预览 API、文件上传 API 与文件下载 API 实现相应的功能，并能够利用 canvas 组件和画布 API 绘制简单的图案。

第5章

微信小程序常用API（下）

学习目标

- ★ 掌握动画 API，能够完成动画的制作
- ★ 熟悉登录流程时序，能够归纳微信小程序的登录流程
- ★ 掌握登录 API，能够运用 wx.login()方法实现用户登录
- ★ 掌握数据缓存 API，能够对数据进行存储、获取和移除等操作
- ★ 掌握头像昵称填写功能，能够实现头像选择和昵称填写
- ★ 掌握腾讯地图 SDK，能够实现腾讯地图 SDK 的接入和使用
- ★ 掌握 map 组件，能够灵活运用 map 组件实现地图效果
- ★ 掌握地图 API，能够实现地图中地理位置的获取等功能
- ★ 掌握位置 API，能够实现获取当前地理位置的功能
- ★ 掌握路由 API，能够利用路由 API 实现页面跳转
- ★ 掌握 WebSocket API，能够成功创建 WebSocket 连接
- ★ 掌握 SocketTask，能够使用 SocketTask 管理 WebSocket 连接

第 4 章讲解了背景音频、录音、音频等 API 的使用，使用这些 API 即可实现相应的功能。本章将继续通过案例的方式讲解微信小程序中常用 API 的使用，主要包括如何使用动画 API 制作动画、如何通过登录 API 实现用户登录、如何调用微信开放能力实现用户头像和昵称的获取、如何通过地图 API 实现地图功能，以及如何通过 WebSocketAPI、SocketTask 实现在线聊天。

【案例 5-1】罗盘动画

在微信小程序中，开发者根据项目需求，可以在页面中添加一些动画效果，例如旋转、缩放、移动等，通过这些动画效果可以提高用户体验。下面以“罗盘动画”微信小程序为例，讲解微信小程序中动画效果的制作。

案例分析

“罗盘动画”微信小程序分为图片区域和按钮区域，实现了对图片的一系列动画操作，包括旋转、缩放、

移动、倾斜等。“罗盘动画”微信小程序的页面效果如图 5-1 所示。

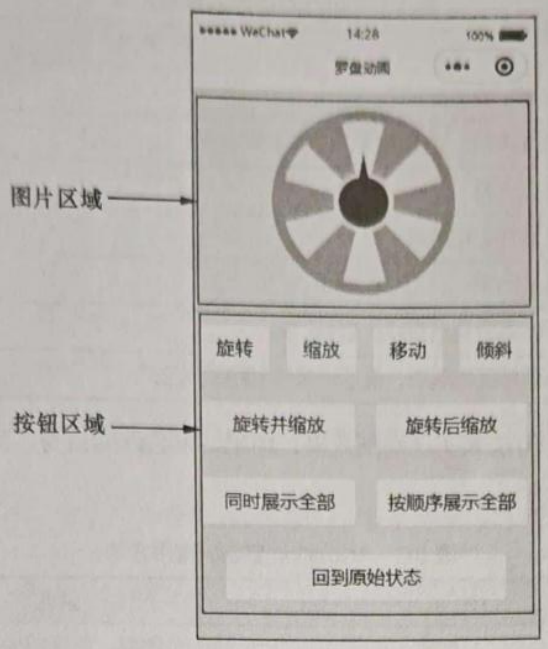


图5-1　“罗盘动画”微信小程序的页面效果

在图 5-1 中，点击不同的按钮，即可实现相应的动画效果。例如，分别点击“旋转”“缩放”“移动”“倾斜”按钮可以实现图片的旋转、缩放、移动、倾斜效果；点击“旋转并缩放”按钮可以实现图片的旋转并缩放效果；点击“旋转后缩放”按钮可以实现图片先旋转，之后再缩放的效果；点击“同时展示全部”按钮可以实现图片同时进行旋转、缩放、移动、倾斜的效果；点击“按顺序展示全部”按钮可以实现图片逐一进行旋转、缩放、移动、倾斜效果；点击“回到原始状态”按钮可以将图片恢复成初始状态。

案例实现

1. 准备工作

在开发本案例前，需要先完成一些准备工作，主要包括创建项目、配置页面、配置导航栏和复制素材，具体步骤如下。

① 创建项目。在微信开发者工具中创建一个新的微信小程序项目，项目名称为“罗盘动画”，模板选择“不使用模板”。

② 配置页面。项目创建完成后，在 app.json 文件中配置一个 compass 页面，具体代码如下。

```
"pages": [
  "pages/compass/compass"
],
```

③ 配置导航栏。在 pages/compass/compass.json 文件中配置页面导航栏，具体代码如下。

```
{
  "navigationBarTitleText": "罗盘动画"
}
```

上述代码将导航栏标题设置为“罗盘动画”。“罗盘动画”导航栏的效果如图 5-3 所示。

④ 复制素材。从本书配套资源中找到本案例，复制以下素材到本项目中。

- pages/compass/compass.wxss 文件，该文件中保存了本项目的页面样式素材。
- images 文件夹，该文件夹保存了本项目所用的素材。

上述步骤操作完成后，“罗盘动画”微信小程序的目录结构如图5-4所示。

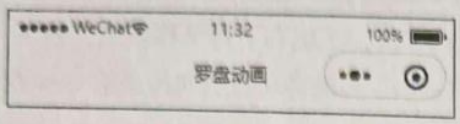


图5-3 “罗盘动画”导航栏的效果

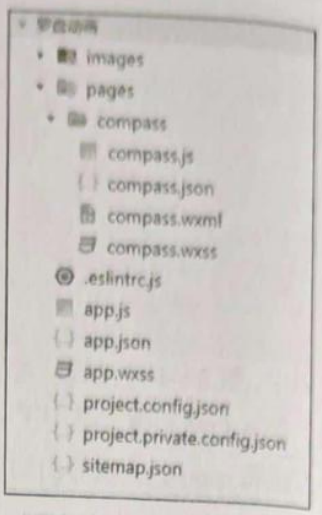


图5-4 “罗盘动画”微信小程序的目录结构

至此，准备工作已经全部完成。

2. 实现“罗盘动画”微信小程序的页面结构

在pages/compass/compass.wxml文件中编写“罗盘动画”微信小程序的页面结构，具体步骤如下。

① 编写图片区域的页面结构，具体代码如下。

```
<view class="anim-pic">
  <image src="/images/img.png" animation="{{ animation }}" />
</view>
```

在上述代码中，第2行代码中的image组件设置了animation属性，通过该属性接收export()方法导出的动画队列数据。

② 编写按钮区域的页面结构，具体代码如下。

```
<view class="anim-btns">
  <button bindtap="rotate">旋转</button>
  <button bindtap="scale">缩放</button>
  <button bindtap="translate">移动</button>
  <button bindtap="skew">倾斜</button>
  <button class="btn-two" bindtap="rotateAndScale">旋转并缩放</button>
  <button class="btn-two" bindtap="rotateThenScale">旋转后缩放</button>
  <button class="btn-two" bindtap="all">同时展示全部</button>
  <button class="btn-two" bindtap="allOrder">按顺序展示全部</button>
  <button class="btn-reset" bindtap="reset">回到原始状态</button>
</view>
```

在上述代码中，第2～10行代码分别绑定了rotate()函数、scale()函数、translate()函数、skew()函数、rotateAndScale()函数、rotateThenScale()函数、all()函数、allOrder()函数、reset()函数，每个函数用于实现不同的功能。

至此，“罗盘动画”微信小程序的页面结构已经实现。

3. 实现“罗盘动画”微信小程序的页面逻辑

在pages/compass/compass.js文件的Page({})中编写页面数据和事件处理函数，实现点击“旋转”“缩放”等不同按钮操作罗盘的功能，具体步骤如下。

① 在pages/compass/compass.js文件中编写页面所需要的数据，具体代码如下。

```
Page({
  data: {
    animation: {}
```

```
  },
})
```

在上述代码中，animation 对象用于保存页面中的动画队列数据。

② 编写 onReady()函数，在页面初次渲染完成时，创建一个动画实例 animation，具体代码如下。

```
onReady: function () {
  this.animation = wx.createAnimation({
    duration: 1000,
    timingFunction: 'ease',
  })
},
```

在上述代码中，第 3 行代码通过将 duration 属性值设置为 1000，实现动画持续时间为 1000 毫秒的效果，第 4 行代码通过将 timingFunction 属性值设置为 ease，实现动画效果为动画以低速开始，然后加快，在结束前变慢的效果。

③ 编写 rotate()函数，实现从原点随机旋转某一个角度的效果，具体代码如下。

```
rotate: function () {
  this.animation.rotate(Math.random() * 720 - 360).step()
  this.setData({
    animation: this.animation.export()
  })
},
```

在上述代码中，第 2 行代码先调用 rotate()方法实现图片的旋转，然后调用 step()方法表示一组动画完成；第 3～5 行代码调用 setData()方法将 export()方法导出的动画队列数据赋值给 animation，实现将动画队列数据传递给 image 组件的 animation 属性的功能。

④ 编写 scale()函数，实现图片随机缩放的效果，具体代码如下。

```
scale: function () {
  this.animation.scale(Math.random() * 2).step()
  this.setData({
    animation: this.animation.export()
  })
},
```

在上述代码中，第 2 行代码调用 scale()方法，实现图片的缩放效果。

⑤ 编写 translate()函数，实现平移变换的效果，具体代码如下。

```
translate: function () {
  this.animation.translate(Math.random() * 100 - 50, Math.random() * 100 - 50).step()
  this.setData({
    animation: this.animation.export()
  })
},
```

在上述代码中，第 2 行代码调用 translate()方法，实现图片的平移变换效果。

⑥ 编写 skew()函数，实现使对象相对 x、y 轴坐标进行随机倾斜，具体代码如下。

```
skew: function () {
  this.animation.skew(Math.random() * 90, Math.random() * 90).step()
  this.setData({
    animation: this.animation.export()
  })
},
```

在上述代码中，第 2 行代码调用 skew()方法，实现图片的随机倾斜效果。

⑦ 编写 rotateAndScale()函数，实现同时进行旋转和缩放，具体代码如下。

```
1 rotateAndScale: function () {
2   this.animation.rotate(Math.random() * 720 - 360)
3   this.animation.scale(Math.random() * 2).step()
4   this.setData({
5     animation: this.animation.export()
6   })
7 },
```

在上述代码中，第2行代码调用rotate()方法，实现图片的旋转；第3行代码调用scale()方法，实现图片的缩放。step()方法是在rotate()方法和scale()方法之后调用的，这样可以实现旋转和缩放同时进行的效果。

⑧ 编写rotateThenScale()函数，实现旋转之后缩放的效果，具体代码如下。

```
1 rotateThenScale: function () {
2   this.animation.rotate(Math.random() * 720 - 360).step()
3   this.animation.scale(Math.random() * 2).step()
4   this.setData({
5     animation: this.animation.export()
6   })
7 },
```

在上述代码中，第2行代码调用rotate()方法，实现图片的旋转；第3行代码调用scale()方法，实现图片的缩放。由于在调用scale()方法前先调用了step()方法，所以缩放会在旋转之后进行。

⑨ 编写all()函数，实现同时展示全部动画，具体代码如下。

```
1 all: function () {
2   this.animation.rotate(Math.random() * 720 - 360)
3   this.animation.scale(Math.random() * 2)
4   this.animation.translate(Math.random() * 100 - 50, Math.random() * 100 - 50)
5   this.animation.skew(Math.random() * 90, Math.random() * 90).step()
6   this.setData({
7     animation: this.animation.export()
8   })
9 },
```

在上述代码中，第2～5行代码通过调用rotate()、scale()、translate()、skew()方法实现了旋转、缩放、平移、倾斜的效果，由于step()方法是在这4个方法之后调用的，所以旋转、缩放、平移、倾斜的效果会同时出现。

⑩ 编写allOrder()函数，实现按顺序展示全部动画，具体代码如下。

```
1 allOrder: function () {
2   this.animation.rotate(Math.random() * 720 - 360).step()
3   this.animation.scale(Math.random() * 2).step()
4   this.animation.translate(Math.random() * 100 - 50, Math.random() * 100 - 50).step()
5   this.animation.skew(Math.random() * 90, Math.random() * 90).step()
6   this.setData({
7     animation: this.animation.export()
8   })
9 },
```

在上述代码中，第2～5行代码通过调用rotate()、scale()、translate()、skew()方法实现了旋转、缩放、平移、倾斜的效果，由于每个方法后面都调用了step()方法，所以这些动画效果会按顺序依次展示。

⑪ 编写reset()函数，实现将动画回到初始状态，具体代码如下。

```
1 reset: function () {
2   this.animation.rotate(0).scale(1).translate(0, 0).skew(0, 0).step({
3     duration: 0
4   })
5   this.setData({
6     animation: this.animation.export()
```

```
  })
}
```

在上述代码中，第 2～4 行代码通过链式调用 rotate()、scale()、translate()、skew()、step()方法将所有方法的参数重置为初始状态。

完成上述代码后，运行程序，“罗盘动画”微信小程序的实现效果如图 5-1 所示。

至此，“罗盘动画”微信小程序已经开发完成。

【案例 5-2】用户登录

在日常生活中，需要用户登录的场景有很多，例如，当用户在手机中浏览文章想要收藏时、在线上购买商品时、进入软件查询个人信息时，只有用户登录自己的账号以后，才可以进一步使用这些功能。下面将讲解如何开发一个具有“用户登录”功能的微信小程序。

案例分析

本案例是一个“用户登录”微信小程序，其初始页面效果如图 5-5 所示。

在图 5-5 中，点击默认头像可以选择用微信头像来作为小程序头像；点击“请输入昵称”输入框可以手动填写昵称或使用微信昵称作为昵称；点击“获取用户的积分”按钮时，如果用户已登录，则会显示对应的积分，如果用户未登录，则会显示“用户不存在，或未登录”。选择头像和填写昵称后的页面效果如图 5-6 所示。

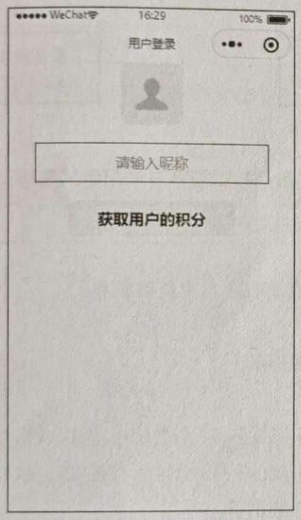


图5-5　初始页面效果

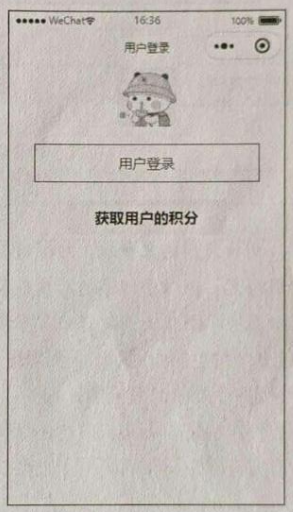


图5-6　选择头像和填写昵称后的页面效果

案例实现

1. 准备工作

在开发本案例前，需要先完成一些准备工作，主要包括创建项目、配置导航栏、复制素材和启动服务器，具体步骤如下。

① 创建项目。在微信开发者工具中创建一个新的微信小程序项目，项目名称为“用户登录”，模板选择“不使用模板”。

② 配置导航栏。在 pages/index/index.json 文件中配置页面导航栏，具体代码如下。

```
{
  "navigationBarTitleText": "用户登录"
}
```

上述代码中配置了导航栏标题为“用户登录”。“用户登录”导航栏的效果如图 5-13 所示。

③ 复制素材。从本书配套资源中找到本案例，复制以下素材到本项目中。

- pages/index/index.wxss 文件，该文件中保存了本项目的页面样式素材。
- images 文件夹，该文件夹保存了本项目所用的素材。

上述步骤操作完成后，“用户登录”微信小程序的目录结构如图 5-14 所示。

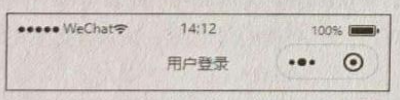


图5-13　“用户登录”导航栏的效果



图5-14　“用户登录”微信小程序的目录结构

④ 启动服务器。从本书配套资源中找到本案例的源代码，进入“服务器端”文件夹，该文件夹下的内容为 Node.js 本地 HTTP 服务器程序。打开 index.js 文件，找到 wx 对象，填写自己的 appid 和 appsecret，具体代码如下。

```
const wx = {
  appid: '填写自己的 appid',
  secret: '填写自己的 appsecret'
}
```

打开命令提示符，切换工作目录到“服务器端”目录下，然后在命令提示符中执行如下命令，启动服务器。

```
node index.js
```

至此，准备工作已经全部完成。

2. 实现用户登录

在 app.js 文件的 App({})中编写代码，实现微信小程序启动时自动执行登录操作，具体步骤如下。

① 定义页面所需的数据，具体代码如下。

```
1 globalData: {
2   token: null // 保存 token
3 },
```

上述代码将 token 属性作为公共数据，用于在多页面中访问。

② 在微信小程序启动时调用 login()方法实现自动登录，具体代码如下。

```
1  onLaunch: function () {
2     this.login()
3  },
4  login: function () {
5    wx.login({
6      success: res => {
7        console.log('login code: ' + res.code)
8        wx.request({
9          url: 'http://127.0.0.1:3000/login',
10         method: 'post',
11         data: {
12           code: res.code
13         },
14         success: res => {
15           console.log('token: ' + res.data.token)
16           // 将 token 保存为公共数据，用于在所有页面中共享 token
17           this.globalData.token = res.data.token
18           // 将 token 保存到数据缓存
19           wx.setStorage({
20             key: 'token',
21             data: res.data.token
22           })
23         }
24       })
25     }
26   })
27 },
```

在上述代码中，第 5～26 行代码通过 wx.login()方法获取登录凭证 code，在成功获取 code 之后，第 8～24 行代码使用 wx.request()方法将 code 发送给服务器。数据请求成功之后，将服务器返回的 token 保存在 globalData 中，用于在多个页面中共享 token。第 19～22 行代码用于将 token 保存到数据缓存中，在后续的开发中将会用到 token。

③ 在微信开发者工具的本地设置中勾选“不校验合法域名、web-view（业务域名）、TLS 版本以及 HTTP 证书”复选框。

上述代码执行后，微信小程序控制台输出结果如图 5-15 所示。

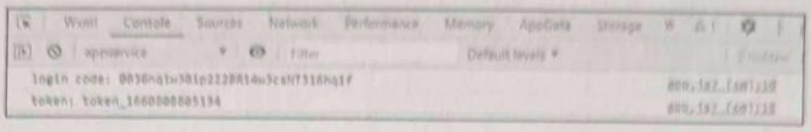


图5-15 微信小程序控制台输出结果

从图 5-15 可以看出，微信小程序已经获得了 login code（微信小程序的登录凭证 code）并收到了服务器返回的 token（自定义登录态），说明登录成功。

查看服务器控制台的输出结果，如图 5-16 所示。

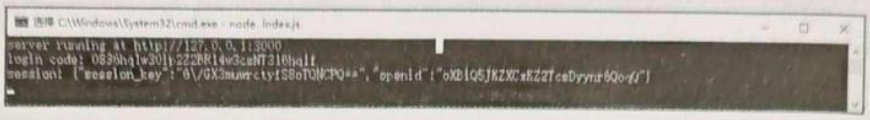


图5-16 服务器控制台的输出结果

从图 5-16 中可以看到 login code（微信小程序的登录凭证 code）和 session（用户会话数据），并且 session 中包含 session_key 和 openid，说明用户登录凭证 code 验证成功。

3. 检查用户是否已经登录

在前面的开发中，微信小程序登录成功后，会将服务器返回的 token 保存在 globalData 和数据缓存中。下次启动微信小程序时，应该先判断 token 是否存在，如果存在，则直接取出这个 token 即可，不用再执行登录操作。需要注意的是，token 有可能会过期，一旦 token 过期就需要重新登录，因此从数据缓存中取出 token 后，应先验证 token 是否过期，再使用 token，具体实现步骤如下。

① 在 app.js 文件的 App({ })中编写 checkLogin()方法，用于判断 token 是否存在，如果 token 存在，则请求服务器，判断 token 是否有效，具体代码如下。

```
1  checkLogin: function (callback) {
2    var token = this.globalData.token
3    if (!token) {
4      // 从数据缓存中获取 token
5      token = wx.getStorageSync('token')
6      if (token) {
7        this.globalData.token = token
8      } else {
9        callback({ is_login: false })
10       return
11     }
12   }
13   wx.request({
14     url: 'http://127.0.0.1:3000/checklogin',
15     data: { token: token },
16     success: res => {
17       callback({ is_login: res.data.is_login })
18     }
19   })
20 },
```

在上述代码中，第 2 行代码用于取出公共数据 token；第 3～12 行代码通过 if 判断 token 是否不存在，如果为 true 则执行第 4～11 行代码，从数据缓存中取出 token，再通过 if 判断，若存在 token，将 token 赋值给 globalData

中的 token，若不存在 token，则调用 callback()函数并传递 is_login 为 false 的参数，第 10 行代码中的 return 表示不执行之后的代码；第 13～19 行代码通过调用 wx.request()方法检查 token 的有效性，通过 res.data.is_login 获取检查的结果，并将检查结果通过参数传递给 callback()函数。

② 修改 app.js 文件中 onLaunch()函数的代码，用于在微信小程序启动后检查用户是否已经登录，如果没有则执行登录操作，具体代码如下。

```
onLaunch: function () {
  this.checkLogin(res => {
    console.log('is_login: ', res.is_login)
    if (!res.is_login) {
      this.login()
    }
  })
},
```

保存上述代码后，运行程序。在数据缓存中已经保存 token 且 token 有效的情况下，会在控制台看到输出结果“is_login: true”，表示用户已经登录。

4. 获取用户的积分

在本案例提供的服务器端程序 index.js 中，会在微信小程序登录成功以后，自动为用户生成初始积分 100，并提供接口 http://127.0.0.1:3000/credit 用于获取积分。接下来将实现获取用户的积分的功能，具体步骤如下。

① 在 pages/index/index.wxml 文件中编写“获取用户的积分”按钮，具体代码如下。

```
<button bindtap="credit">获取用户的积分</button>
```

上述代码中，为 button 组件绑定 tap 事件，事件处理函数为 credit()。

② 在 pages/index/index.js 文件中编写代码，具体代码如下。

```
const app = getApp()
Page({
  credit: function () {
    wx.request({
      url: 'http://127.0.0.1:3000/credit',
      data: {
        token: app.globalData.token
      },
      success: res => {
        console.log(res.data)
      }
    })
  },
})
```

在上述代码中，第 1 行代码通过 getApp()函数获取 App 实例，从而获取 app.js 文件中定义的公共数据 globalData；第 3～13 行代码定义了 credit()方法，用于调用 wx.request()方法向服务器发送请求，如果请求成功，就会在控制台中输出服务器的返回结果。

保存上述代码后，运行程序。等待控制台中输出了 token 后，单击“获取用户的积分”按钮，即可在控制台中看到服务器返回的用户积分，如图 5-17 所示。

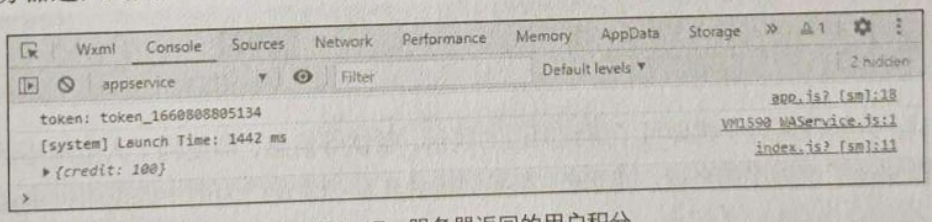


图5-17　服务器返回的用户积分

当用户未登录时，单击“获取用户的积分”按钮则无法获取用户的积分。为了演示用户未登录的情况，可以临时将 onLaunch()函数注释起来，然后重新运行微信小程序，控制台中输出的结果如图 5-18 所示。

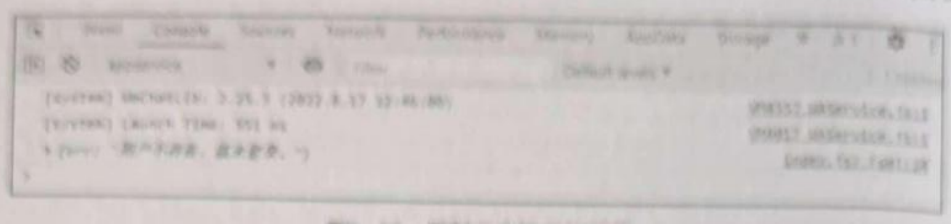


图5-18 控制台中输出的结果

从图 5-18 可以看出，服务器返回结果为“用户不存在，或未登录。”，说明当前用户未登录，无法获取用户的积分。

5. 获取用户头像和昵称

在微信小程序中可以通过头像昵称填写功能来获取微信用户头像和昵称，具体步骤如下。

① 在 pages/index/index.js 文件中定义页面所需的数据，具体代码如下。

```
1 const defaultAvatar = '/images/avatar.png'
2 Page({
3   data: {
4     avatarUrl: defaultAvatar,
5   },
6   原有代码……
7 })
```

在上述代码中，第 1 行代码定义的 defaultAvatar 为默认头像路径地址；第 4 行代码定义了 avatarUrl 属性，表示头像路径地址。

② 在 pages/index/index.wxml 文件中，找到“获取用户的积分”按钮的代码，在其上方编写页面结构代码，具体代码如下。

```
1 <button class="avatar-wrapper" open-type="chooseAvatar" bindchooseavatar="onChooseAvatar">
2   <image class="avatar" src="{{ avatarUrl }}" />
3 </button>
4 <input type="nickname" class="nickname" placeholder="请输入昵称" />
```

在上述代码中，第 1～3 行代码定义了 button 组件和 image 组件，用于选择头像和展示头像；第 4 行代码定义了 input 组件，用于输入昵称。

③ 在 pages/index/index.js 文件中编写 onChooseAvatar()函数，获取头像信息的临时路径，具体代码如下。

```
1 onChooseAvatar: function (e) {
2   console.log(e)
3   const { avatarUrl } = e.detail
4   this.setData({ avatarUrl })
5 },
```

保存上述代码后，运行程序，测试是否可以正确获取用户头像和昵称。

至此，“用户登录”微信小程序已经开发完成。

【案例 5-3】查看附近美食餐厅

信息化时代，人们仅通过手机即可了解到周边的娱乐、餐饮等信息，即使到了人生地不熟的地方，也可以通过手机地图快速了解周围的环境。接下来，本案例将会开发一个专门为用户提供附近美食餐厅信息的微信小程序。

案例分析

“查看附近美食餐厅”微信小程序打开后，会显示一个地图组件，并提供一些功能按钮，如图 5-19 所示。在图 5-19 中，页面上方有个 banner 图，用于显示优惠券领取信息；地图左下角有一个定位按钮，点击此按钮可以将地图中心点设为当前定位的位置。点击 banner 图会跳转到优惠券页面，优惠券页面如图 5-20 所示。



图5-19　“查看附近美食餐厅”微信小程序



图5-20　优惠券页面

从图 5-20 可以看出，当前有两个优惠券，都是“已领取”的状态。

案例实现

1. 准备工作

在开发本案例前，需要先完成一些准备工作，主要包括创建项目、配置页面、配置导航栏和复制素材，具体步骤如下。

① 创建项目。在微信开发者工具中创建一个新的微信小程序项目，项目名称为“查看附近美食餐厅”，模板选择“不使用模板”。

② 配置页面。在 app.json 文件中配置两个页面，具体代码如下。

```
"pages": [
  "pages/map/map",
  "pages/coupon/coupon"
],
```

③ 配置导航栏。在 pages/map/map.json 文件中配置页面导航栏，具体代码如下。

```
{
  "navigationBarTitleText": "查看附近美食餐厅",
  "navigationBarBackgroundColor": "#d9362c",
  "navigationBarTextStyle": "white"
}
```

在上述代码中，设置导航栏标题为“查看附近美食餐厅”，导航栏背景色为#d9362c，导航栏文字颜色为 white。“查看附近美食餐厅”导航栏的效果如图 5-31 所示。

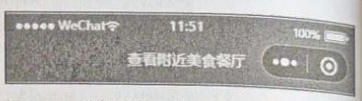


图5-31　“查看附近美食餐厅”导航栏的效果

在 pages/coupon/coupon.json 文件中配置页面导航栏，具体代码如下。

```
{
  "navigationBarTitleText": "优惠券",
  "navigationBarBackgroundColor": "#d9362c",
  "navigationBarTextStyle": "white"
}
```

在上述代码中，设置导航栏标题为“优惠券”，导航栏背景色为#d9362c，导航栏文字颜色为 white。“优惠券”导航栏的效果如图 5-32 所示。

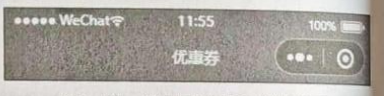


图5-32　“优惠券”导航栏的效果

④ 复制素材。从本书配套资源中找到本案例，复制以下素材到本项目中。

- pages/map/map.wxss 文件，该文件中保存了“查看附近美食餐厅”页面的样式素材。
- pages/coupon/coupon.wxss 文件，该文件中保存了“优惠券”页面的样式素材。

- libs 文件夹，该文件夹保存了微信小程序 JavaScript SDK。
- images 文件夹，该文件夹保存了本项目所用的图片素材。

按照上述步骤操作完成后，“查看附近美食餐厅”微信小程序的目录结构如图 5-33 所示。

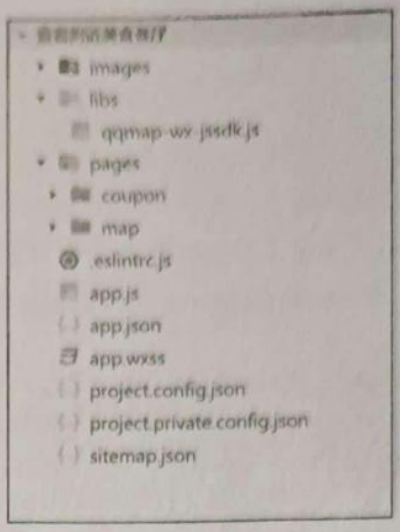


图5-33　“查看附近美食餐厅”微信小程序的目录结构

至此，准备工作已经全部完成。

2. 获取初始数据

在 pages/map/map.js 文件中编写页面所需的数据，具体代码如下。

```
1  const key = '填写自己申请的开发者密钥'
2  const QQMapWX = require('../../libs/qqmap-wx-jssdk.js')
3  const qqmapsdk = new QQMapWX({ key })
4  Page({
5    data: {
6      scale: 18,          // 缩放
7      longitude: 0,       // 地图中心点经度
8      latitude: 0,        // 地图中心点纬度
9      markers: []         // 标记点
10   },
11   mapCtx: null,
12 })
```

在上述代码中，第 1 行代码定义了 key 属性，表示腾讯地图 SDK 开发者密钥，为必填项；第 3 行代码定义了 qqmapsdk，用于保存 QQMapWX 实例；第 11 行代码定义了 mapCtx 属性，用于保存 MapContext 实例。

3. 在页面中显示地图

在地图界面上会显示中心点的图片和附近美食餐厅的图片，下面分别进行实现。

① 在 pages/map/map.wxml 文件中编写页面结构，具体代码如下。

```
1 <map id="myMap" bindregionchange="regionChange" longitude="{{ longitude }}" latitude="{{ latitude }}" markers="{{ markers }}" scale="{{ scale }}" show-location></map>
2 <view class="controls">
3   <image class="banner" mode="widthFix" src="/images/banner.png" bindtap="bannerTap" />
4   <image class="gps" src="/images/gps.png" bindtap="controlTap" />
5 </view>
```

在上述代码中，第 1 行代码通过 map 组件来实现地图界面，其中 regionChange()函数会在视野发生变化时触发，longitude 属性用于设置地图中心点经度，latitude 属性用于设置地图中心点纬度，markers 属性用于设置标记点，scale 属性用于设置缩放级别；第 3 行代码用于在地图上方显示 banner 图；第 4 行代码用于在地图上方显示定位按钮。

② 在 pages/map/map.js 文件中编写 onLoad()函数，实现页面加载时获取当前位置的经度和纬度，具体代码如下。

```
1 onLoad : function () {
2   wx.getLocation({
3     type: 'gcj02',
4     success: res => {
5       this.setData({
6         longitude: res.longitude,
```

```
        latitude: res.latitude
      })
    }
  })
},
```

在上述代码中，第2～10行代码调用了wx.getLocation()方法，获取当前的地理位置（经度、纬度），其中第3行代码定义了type属性，属性值为gcj02，表示地图使用的坐标格式。

③ 在pages/map/map.js文件中编写onReady()函数，实现页面初次渲染时获取MapContext实例，具体代码如下。

```
onReady: function () {
  this.mapCtx = wx.createMapContext('myMap')
},
```

在上述代码中，第2行代码调用wx.createMapContext()方法获取mapContext实例。

④ 在app.json文件中声明其需调用的地理位置相关接口，具体代码如下。

```
"permission": {
  "scope.userLocation": {
    "desc": "你的位置信息将用于小程序位置接口的效果展示"
  }
},
"requiredPrivateInfos": [
  "getLocation"
]},
```

在上述代码中，scope.userLocation用于对位置权限进行声明；desc用于微信小程序获取权限时展示接口用途说明。

4. 实现跳转到优惠券页面

在pages/index/index页面点击优惠券，实现跳转到优惠券页面的功能，具体步骤如下。

① 在pages/map/map.js文件中编写bannerTap()函数，具体代码如下。

```
bannerTap: function () {
  wx.navigateTo({
    url: '/pages/coupon/coupon'
  })
},
```

在上述代码中，第2～4行代码调用wx.navigateTo()方法，实现跳转到优惠券页面，其中，第3行代码中的url为需要跳转的路径。

② 在pages/coupon/coupon.wxml文件中编写优惠券页面的页面结构，具体代码如下。

```
<view class="coupon">
  <image src="/images/couponone.png" />
</view>
<view class="coupon">
  <image src="/images/coupontwo.png" />
</view>
```

在上述代码中，第1～3行代码定义了第1张优惠券，其中第2行代码定义了image组件，用于显示优惠券的图片；第4～6行代码定义了第2张优惠券，其结构与第1张优惠券类似。

5. 实现查找附近美食餐厅功能

“查看附近美食餐厅”微信小程序可以根据当前位置（经度和纬度）搜索附近的餐厅，并将其通过标记点标记出来，具体步骤如下。

① 在pages/map/map.js文件的Page({ })中编写getFood()函数，实现查找附近美食餐厅的功能，具体代

码如下。

```
getFood: function (longitude, latitude) {
  // 调用接口
  qqmapsdk.search({
    // 搜索关键词
    keyword: '餐厅',
    location: {
      longitude: longitude,
      latitude: latitude
    },
    success: res => {
      var markers = []
      // 为附近的美食餐厅设置标记点
      for (let i in res.data) {
        markers.push({
          iconPath: '/images/food.png',
          id: markers.length,
          latitude: res.data[i].location.lat,
          longitude: res.data[i].location.lng,
          width: 30,
          height: 30
        })
      }
      markers.push({
        iconPath: '/images/center.png',
        id: res.data.length,
        latitude: latitude,
        longitude: longitude,
        width: 15,
        height: 40
      })
      // 将搜索结果显示在地图上
      this.setData({ markers })
    }
  })
},
```

在上述代码中，第3～34行代码调用了qqmapsdk对象的search()方法，其中第5代码定义了搜索关键词“餐厅”；第6～9行代码定义了位置坐标location，包含经度和纬度；第10～33行代码定义了success()回调函数，表示接口调用成功。在回调函数中，第11行代码定义了markers标记数据，用于存放位置信息，第13～22行代码通过for循环将获取到的附近美食餐厅的位置信息通过push()方法保存在markers数组中；第23～30行代码用于显示地图中心点；第32行代码用于将搜索结果显示在地图上。

② 在pages/map/map.js文件中编写regionChange()函数，实现地图移动时更新地图上的标记点，具体代码如下。

```
regionChange: function (e) {
  if (e.type === 'end') {
    this.mapCtx.getCenterLocation({
      success: res => {
        this.getFood(res.longitude, res.latitude)
      }
    })
  }
```

```
9 },
```

在上述代码中，第 3～7 行代码通过调用 getCenterLocation()方法，实现获取当前地图中心的经纬度，接口调用成功之后，调用 getFood()方法获取当前位置附近的美食餐厅。

③ 在 pages/map/map.js 文件中编写 controlTap()函数，具体代码如下。

```
1 controlTap: function (e) {
2   this.mapCtx.moveToLocation()
3 }
```

在上述代码中，第 2 行代码调用 moveToLocation()方法，将地图中心移动到当前定位点。

④ 在微信开发者工具的本地设置中勾选“不校验合法域名、web-view（业务域名）、TLS 版本以及 HTTPS 证书”复选框。

完成上述步骤后，运行程序，在地图上可以看到附近的美食餐厅的图标了。

至此，“查看附近美食餐厅”微信小程序已经开发完成。

【案例 5-4】在线聊天

一些购物平台经常会通过人工客服或机器客服来为用户提供服务，及时解答用户提出的问题，提高用户的使用体验。对于这类需求，可以通过在微信小程序中开发在线聊天功能来实现。下面将对“在线聊天”微信小程序进行详细讲解。

案例分析

“在线聊天”微信小程序初始页面效果如图 5-34 所示。

在图 5-34 的页面底部的输入框中输入“我来自微信小程序端”文本后，点击“发送”按钮，该页面就会将用户输入的内容发送给服务器，并将用户输入的内容显示在聊天界面的右侧。当服务器收到消息后，会自动回复一条消息“自动回复”，显示在聊天界面的左侧。微信小程序向服务器发送消息的页面效果如图 5-35 所示。

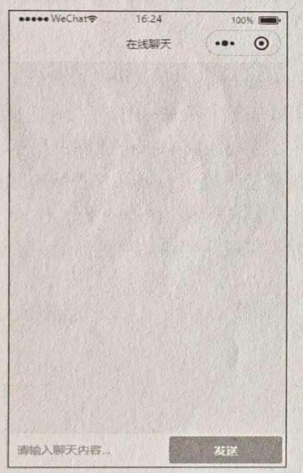


图5-34 初始页面效果

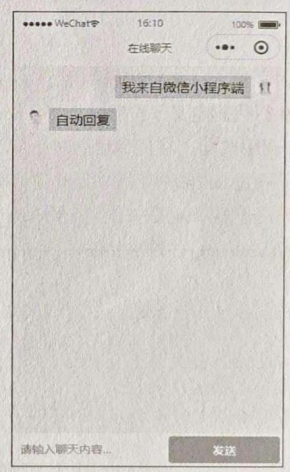


图5-35 微信小程序向服务器发送消息的页面效果

在服务器接收到消息后，在控制台中输出消息的效果如图 5-36 所示。

在服务器的控制台中输入的消息，也可以发送给微信小程序。例如，在服务器的控制台中输入“我来自服务器端”文本后，按回车键，控制台输出消息的效果如图 5-37 所示。

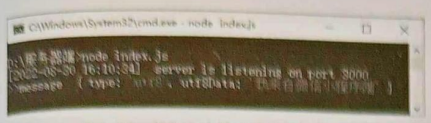


图5-36 控制台中输出消息的效果

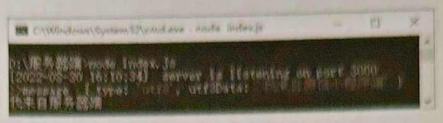


图5-37 控制台输出消息的效果

微信小程序端接收消息的页面效果如图 5-38 所示。

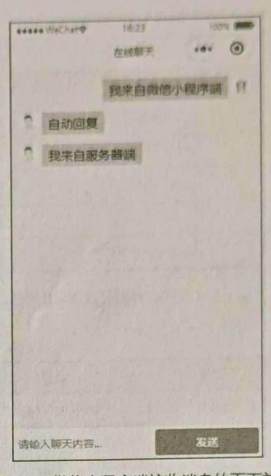


图5-38 微信小程序端接收消息的页面效果

具体步骤如下。

① 创建项目。在微信开发者工具中创建一个新的微信小程序项目，项目名称为“在线聊天”，模板选择“不使用模板”。

② 配置导航栏。在 pages/index/index.json 文件中配置页面导航栏，具体代码如下。

```
{
  "navigationBarTitleText": "在线聊天"
}
```

上述代码将导航栏标题设置为“在线聊天”。“在线聊天”页面导航栏的效果如图 5-39 所示。

③ 复制素材。从本书配套资源中找到本案例，复制以下素材到本项目中。

- index.wxss 文件，该文件中保存了本项目的页面样式素材。
- images 文件夹，该文件夹保存了本项目所用的图片素材。

上述步骤完成后，“在线聊天”微信小程序的目录结构如图 5-40 所示。

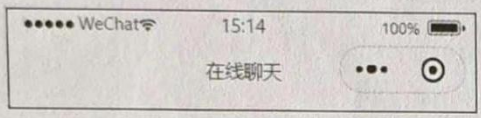


图5-39 “在线聊天”页面导航栏的效果

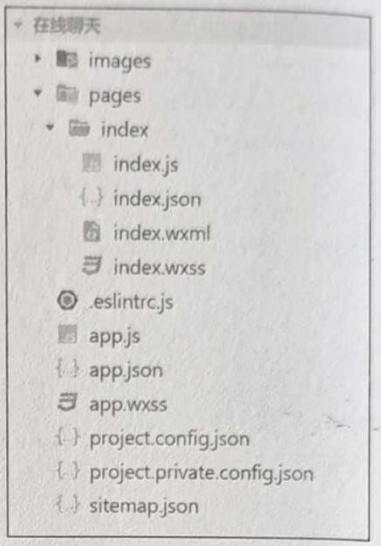


图5-40 “在线聊天”微信小程序的目录结构

④ 启动服务器。从本书配套资源中找到本案例的源代码，进入“服务器端”文件夹，该文件夹下的内容为 Node.js 本地 HTTP 服务器程序。打开命令提示符，切换工作目录到当前目录，然后在命令提示符中执行如下命令，启动服务器。

```
node index.js
```

至此，准备工作已经完成。

2. 实现 WebSocket 连接

实现“在线聊天”微信小程序的 WebSockt 连接，具体步骤如下。

① 在 pages/index/index.js 文件的 Page({ })中编写 onLoad( )函数，具体代码如下。

```
ws: null,
onLoad: function () {
  const ws = wx.connectSocket({
    url: 'ws://127.0.0.1:3000',
    success: resConnect => {
      console.log(resConnect)
    },
    fail: resConnectError => {
      console.log(resConnectError)
    }
```

```
11  })
12  ws.onMessage(data => {
13    console.log(data)
14  })
15  ws.onClose(res => {
16    console.log(res)
17  })
18  this.ws = ws
19 },
```

在上述代码中，第 3～11 行代码调用 wx.connectSocket()方法连接 WebSocket；第 12～14 行代码调用 onMessage()方法监听 WebSocket 连接接收到服务器的消息事件，表示服务器返回了消息；第 15～17 行代码调用 onClose()方法来监听 WebSocket 连接关闭事件；第 18 行代码将通过 wx.connectSocket()方法创建连接成功之后的返回值 WebSocket 保存在 ws 中，让整个页面中可以使用。

② 在 pages/index/index.js 文件中编写 onUnload()函数，实现关闭 WebSocket 连接，具体代码如下。

```
1 onUnload: function () {
2   this.ws.close()
3 },
```

③ 在微信开发者工具的本地设置中勾选“不校验合法域名、web-view（业务域名）、TLS 版本以及 HTTPS 证书”复选框。

保存上述代码，运行程序，在微信小程序的控制台中会看到图 5-41 所示的效果。

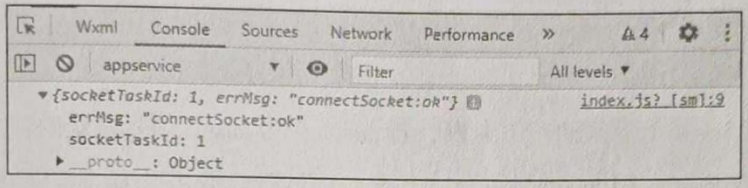


图5-41　控制台中输出的消息

图 5-41 中控制台中输出的消息说明 WebSocket 连接成功。

3. 实现发送消息功能

WebSocket 连接成功后，下面实现将消息发送给服务器的功能，具体步骤如下。

① 在 pages/index/index.wxml 文件中编写页面结构，具体代码如下。

```
1 <view class="chat">
2   <!-- 消息列表 -->
3 </view>
4 <view class="message">
5   <input type="text" bindinput="input" placeholder="请输入聊天内容..." value="{{ content }}" />
6   <button type="primary" bindtap="send" size="small">发送</button>
7 </view>
```

在上述代码中，第 1～3 行代码定义了消息列表区域，该区域中消息的展示在之后的步骤中实现。第 4～7 行代码定义了底部消息发送区域，其中第 5 行代码定义了 input 组件，用于输入聊天内容，绑定 input 事件，该事件在键盘输入时触发，事件处理函数为 input()；第 6 行代码定义了 button 组件，该组件表示“发送”按钮，绑定 tap 事件，事件处理函数为 send()。

② 在 pages/index/index.js 文件的 Page({})中编写页面中所需的数据，具体代码如下。

```
data: {
  content: ''
},
message: '',
```

在上述代码中，content 表示页面中展示的消息内容；message 表示输入框中输入的内容，即需要发送的内容。

③ 在 pages/index/index.js 文件的 Page({ })中编写 input()函数，实现将输入框输入的文本保存到 message 中，具体代码如下。

```
input: function (e) {
  this.message = e.detail.value
},
```

④ 在 pages/index/index.js 文件的 Page({ })中编写 input()函数，实现点击“发送”按钮时将 message 中的内容发送到服务器，具体代码如下。

```
1  send: function () {
2    // 判断发送内容是否为空
3    if (!this.message) {
4      wx.showToast({
5        title: '消息不能为空',
6        icon: 'none',
7        duration: 2000
8      })
9      return
10   }
11   // 发送消息到服务器
12   this.ws.send({
13     data: this.message
14   })
15 },
```

在上述代码中，第 3～10 行代码进行 if 判断，若 message 为 false，表示发送内容为空，则弹出消息提醒“消息不能为空”，若 message 为 true，则执行 12～14 行的代码。第 12～14 行代码调用 send()方法，通过 WebSocket 连接将 message 中的内容发送给服务器。

保存上述代码，运行程序，在微信小程序的输入框中输入“您好。服务器”。服务器的控制台输出的结果，如图 5-42 所示。

图 5-42 中，服务器已经成功接收到了微信小程序发送的字符串信息“您好。服务器”。

在微信小程序的控制台会输出图 5-43 所示的消息。

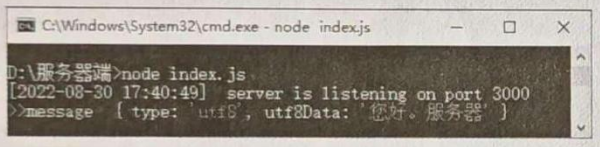


图5-42 服务器的控制台输出的结果

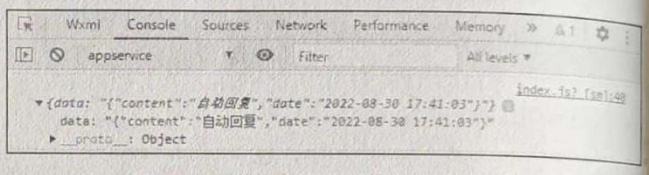


图5-43 微信小程序的控制台输出的消息

从图 5-43 可以看出，微信小程序收到了服务器自动回复的消息。

4. 实现消息列表展示功能

前面实现了服务器和微信小程序互相发送消息的功能，接下来需要将双方的消息列表展示在页面中，具体步骤如下。

① 在 pages/index/index.js 文件的 Page({ })的 data 中增加 list 数组，具体代码如下。

```
list: [],
lastId: ''
```

在上述代码中，list 表示消息列表；lastId 表示最后一条消息的 id。

② 在 pages/index/index.js 文件中，修改 send()方法，实现点击“发送”按钮时将发送的内容存入 list 数

组，同时将content、message恢复初始值，具体代码如下。

```
send: function () {
  原有代码……
  const list = this.data.list
  const lastId = list.length
  list.push({
    id: lastId,
    content: this.message,
    role: 'me'
  })
  this.setData({ list, lastId, content: '' })
  this.message = ''
}
```

在上述代码中，第3行代码将data中的list数组取出来，表示消息列表；第4行代码将list数组的长度保存为lastId；第5～9行代码将id、content、role这些数据通过push()方法写入list数组中，其中role为角色，它的值'me'表示该条消息是由微信小程序端发送的，第10行代码调用setData()方法对list、lastId、content进行更新，第11行代码清空了this.message。

③ 在pages/index/index.js文件中修改ws.onMessage()方法，实现在收到服务器端传来的消息时对数据进行处理，具体代码如下。

```
ws.onMessage(msg => {
  const data = JSON.parse(msg.data)
  const list = this.data.list
  const lastId = list.length
  list.push({
    id: lastId,
    content: data.content,
    role: 'server'
  })
  this.setData({ list, lastId })
})
```

在上述代码中，第2行代码将接收到的数据转换成JavaScript对象；第5～9行代码将id、content、role写入list数组，其中role的值为'server'表示该条消息是由服务器端发送的。

④ 在pages/index/index.wxml文件中编写页面结构，用于展示消息列表，具体代码如下。

```
<scroll-view scroll-y scroll-into-view="item_{{ lastId }}">
  <view wx:for="{{ list }}" wx:key="id" class="chat-message chat-message-{{ item.role }}"
id="item_{{ item.id }}">
    <view class="chat-content"><view>{{ item.content }}</view></view>
    <image class="chat-avatar" src="/images/{{ item.role }}.png" />
  </view>
</scroll-view>
```

在上述代码中，第1～6行代码定义了scroll-view组件，用于当消息数大于页面显示高度时，可以纵向滚动显示消息列表。其中，第2行代码通过wx:for控制属性将list数组中的消息在页面上进行渲染；第3行代码定义了view组件，用于展示消息内容；第4行代码定义了image组件，用于根据角色展示图片。

完成上述代码后，运行程序，测试聊天功能是否正确，“在线聊天”微信小程序的实现效果如图5-35～图5-38所示。

至此，“在线聊天”微信小程序已经开发完成。